

《七层塔》

一部关于 AI Agent Harness 进化的非虚构小说
写给想看清 "prompt 工程师" 下一步是什么的人

前言 · 这本书是怎么写出来的

2027 年 4 月的一个晚上，我合上备忘录，把刚写完的《七层塔》第七章拷到飞书，发了一个朋友。

朋友读完后回了三个字："是 AI 写的吧？"

我想了想，回答："是我写的，AI 帮我盖的。"

这句话就是这本书前言的全部。

一、本书的来历

这本书的起点是 2026 年 9 月 3 日早上，一个想法：

"我们本地存了这么多 *llm-wiki* 的知识，能否综合后写一本给我看的科普小说，可以把知识串起来？"

我手头有一份 "Karpathy-style 知识库"——1,033 个知识节点（209 个概念 + 166 个实体 + 658 个信源），全部维护在 `C:\Users\Administrator\clawd\memory\wiki\` 下，干净、规整、有索引。

这个 wiki 的主轴是 **AI Agent Harness 工程**——从 prompt、tools、memory、skills、subagents、evaluation、self-improvement，七层架构，过去三年的真实项目。

把这些散点串成 "故事"，是我一直想做但没做的事。

写小说看起来是创意活，本质是**结构化知识重构**——把零散的事实压进一条叙事线索，让读者在故事里 "看见" 知识之间的关联。这正是 AI 擅长的活。

所以我开始干。

二、本书的写作流程

我没有用一个大模型写 5 万字。

我用的是"读者-作者"循环：

1. 我读 wiki 主轴，搭出 7 层 Harness 框架
2. 我给读者 4 个故事结构选项（七层塔 / 编年史 / 双线 / 聚焦 Memory）
3. 读者选 "七层塔"
4. 我写序章 + 第一章（约 6,000 字），用 media tag 交付
5. 读者给 3 个维度的反馈（节奏 / 范围 / 事实准确度）
6. 我按反馈连写第二章到第四章
7. 循环直到全书写完

每一章都先给读者看，再写下一章。读者不是被动接受者，是"实时编辑"。

这种工作流对 AI 写作是必要的——AI 一次写 5 万字，几乎一定会有"偏题 / 重复 / 走神"的问题。**分章 + 即时反馈**是质量的护城河。

三、本书的"事实优先"原则

读者在第一轮反馈里选了"事实优先"。

这意味着：

- 每个项目 / 论文 / 文档都要有真实出处
- 价格、时间、GitHub Star 数都查了真实数据
- 不确定的事件，宁可 not write

每章末尾有一个"引用"段，列出本章涉及的真实 wiki 节点。这部分不是装饰——是"如果你对某个项目感兴趣，从这里开始"的入口。

小说里的"事实"和 wiki 里的"事实"严格对齐。这是这本书跟"AI 写的其他小说"最大的不同——它可以被当作一份带叙事的工程参考。

四、本书的局限

这本书不适合以下场景：

- **零基础读者**——它假设你懂"什么是 LLM"、"什么是 API"
- **只看故事的读者**——它有 60+ 个技术引用，密度比一般小说高
- **想读"未来"的人**——它写的是 2024-2027 的事，不是 2030+ 的预言

这本书**适合**以下场景：

- 工程师 / 产品经理 / 研究员 / 投资人 / 学生
- 想知道"AI agent 工程的完整链路"的人
- 想知道"这 3 年 AI 行业怎么从 5 行 prompt 走到 self-improvement"的人
- 想"重新发现自己手上在做的事的坐标"的人

如果你符合最后一条，这本书就是写给你的。

五、致谢

这本书的真正作者不止我。

给读者：谢谢你选"七层塔"，选"事实优先"，选"写 L5+L6 两章"。每一次选择都让这本书更接近你想要的形状。

给知识库：clawd/memory/wiki 是过去 3 年积累的 803 页 / 2,138 个 wiki 链接的"行业地图"。没有这张地图，这本书写不出来。

给每一个被引用的项目：Anthropic / OpenAI / DeepSeek / Sakana AI / Stanford / 腾讯程序员 / Mem0 / Multica / HiClaw / Ruflo / 深即达 / Skillify / ai-memory / teamai-cli / Lilian Weng / 孙敦灿 / 公众号「关注前沿的」 / 公众号「开源星探」

是你们造了 L1-L7。

是你们让这本书有了锚点。

— 2027 年 4 月，深圳

《七层塔》

一部关于 AI Agent Harness 进化的非虚构小说
——写给想看清"prompt 工程师"下一步是什么的人

序章 · 华强北，2024 年 3 月

深圳华强北，科兴科学园 B 座 11 楼。

李明盯着屏幕右下角的 404 Not Found，又过去 12 分钟了。他三个月前在这家 SaaS 公司做后端，3 月 1 号被 HR 一通电话"优化"——N+1 的工资当天下午就到账，比他写的最后一个 bugfix 还快。剩下 23 个工作日的年假，公司说"不用休，给您折现"。他算了算，按基本工资折，到手不到 4 万，于是选了休。

休假的第一个礼拜他没出门，把阳台上的多肉浇了三次水，第二次浇完才意识到上一次才过了两天。第三次浇完，他决定出门。

"科兴"方圆 500 米内最像样的去处是负一层的瑞幸。他本来只想买杯美式，结果撞上了一场"AI 编程线下分享会"——免费、扫码入场、还送杯卡。主持人是个二十出头的男孩，工牌上写着"Trae · 字节跳动"。

台上那个人讲了 40 分钟，PPT 上 90% 的内容李明都听懂了：他之前做后端用 Java + Spring Boot，GPT-4 的 API 文档比 Spring 的简单十倍，剩下的就是改改 prompt。但剩下 10% 他没听懂——

"当你把 prompt 当作软件工程来对待，它就是软件工程。"

散场后李明加了分享者的微信，对方回得很快："哥，你可以先去玩玩 Anthropic 那个 Console，免费的，能看 token 怎么算。"他还补了一句："你是程序员吧？那你看 Prompt Caching 那段就行，2024 年 3 月刚发的，省 90% 成本那个。"

回出租屋的地铁上，李明刷到了那条推文。Anthropic 写道：Prompt Caching reduces latency by >2x and cost by up to 90%. 他把"90%"看了三遍，想起自己最后一次 deploy 的 SpringBoot 服务，把 5 个小时的 Java 启动时间优化到 40 秒。优化 90% 用了一个月。

到站的时候他在手机上下载了 Claude 的 app。他想：**我能不能做点什么？**

那天晚上，他写下了自己第一行 prompt。

```
import anthropic

client = anthropic.Anthropic()
msg = client.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=1024,
    system="你是一个专业的邮件助手。",
    messages=[
        {"role": "user", "content": "帮我把这段话改成正式一点的商务语气: hi 那个 demo 我下周三才"}
    ]
)
print(msg.content[0].text)
```

跑通了。输出："您好，由于本周日程较为紧张，该 demo 将延期至下周三上线，对此带来的不便深表歉意。"

李明盯着那行字看了十几秒。他以前做后端，要写一个能改写语气的小工具，至少要：(1) 找 NLU 服务或自己训模型，(2) 写个 Flask API 包一层，(3) 部署到一台 1 核 2G 的云服务器，(4) 监控日志、排 nginx、配 HTTPS。全套下来三天起步。

现在：5 行代码，4 秒钟。

他忽然明白台上那个人说的是什么意思——**这 5 行不是 demo，是地基。**

那天他在备忘录里写下：

2024-03-23

想做一个"AI 写周报"的 SaaS。

思路：用户填本周做了什么 → GPT 改写成周报 → 可选调语气。

风险：同质化严重，怎么差异化？

灵感：先做 100 个 prompt，让用户选。

他看着"100 个 prompt"四个字，没有删。

第一章 • L1 Prompt — "你只要会说话"

1.1 三个 prompt 决定一个 SaaS

四月中旬，李明的"AI 周报助手"上线了。

版本号 0.1，没写登录系统，没用数据库，prompt 写在 Python 文件的字符串里。唯一的差异化是**风格选择器**——他用 Three.js 做了三个按钮，"老板向 / 平级向 / 客户向"，每个按钮下面是一个手写的 prompt。

他后来回想，这是他人生中第一次体会到**prompt 工程的全部诅咒和祝福**——

老板向 prompt 长这样：

你是一位十年经验的产品经理，正在向上级领导做周报汇报。

请用以下结构改写用户的输入：

1. 本周核心成果（不超过 3 条）
2. 关键数据 / 进度
3. 风险与需要支持
4. 下周计划

风格要求：

- 主动语态，少用被动
- 数字必须保留
- 不要使用"我们认为"、"我个人觉得"等弱化主语的表达
- 结尾加一句"如需进一步说明，请随时联系我"

输入：

```
{user_input}
```

平级向 prompt 大同小异，但把"上级"换成"同组同事"，把"主动汇报"换成"信息同步"。客户向最简单——把"我们"全换成对方公司名，加一句"欢迎随时沟通"。

跑了一周，李明用 Cursor 写的运营脚本统计发现：**老板向的留存是平级向的 3 倍**。他一开始以为是因为老板向付费意愿强，后来才明白真相：老板向的 prompt 是他**写了 11 版才稳定**的，平级向只写了 2 版。

那 11 版里他学会了几个名字——

- **温度 (temperature)**：从 1.0 调到 0.3，输出不再飘
- **系统提示 (system prompt)**：把"角色 + 风格 + 输出结构"分开，比塞在一个 user message 里稳定 10 倍

- **少样本 (few-shot)**: 他加了 3 个例子之后, 输出格式再没跑偏过
- **XML / JSON 输出**: 他试过用 "请用 JSON 输出", 失败率 40%; 改成 "请将改写后的内容放在 `json` 代码块里", 失败率 5%

这四个词, Lilian Weng 在 2023 年 3 月那篇《Prompt Engineering》长文里都讲过。李明没读过那篇文章——他是自己撞墙撞出来的。

但他撞出来的成本是 **2 周**。Lilian Weng 用 1 万字告诉读者同样的事。

1.2 提示工程是 2024 年最小的工程

2024 年 5 月, 李明的周报助手日活爬到了 800。同期 Anthropic 在 5 月 9 日发布《Prompt Engineering Overview》——一份 30 页的官方指南, 主题就是李明这 2 周干的事。

The Anthropic Console now offers a Prompt Generator tool that writes prompts for you based on a few examples and a desired output... The tool runs against Claude 3 Haiku, the fastest and most affordable model in the Claude 3 family.

李明读完那份指南, 笑了。

他笑的不是内容——内容他已经全都会了。他笑的是**Anthropic 把"会写 prompt"做成了产品功能**。意思是: 他 2 周学的东西, Anthropic 认为它应该被工具化。

后来这件事反复发生。OpenAI 在 2024 年 8 月推出 Structured Outputs, 把 JSON 输出的失败率从 5% 砸到 < 1%; Google 在 2024 年 9 月推出 Gemini 的 System Instructions, 把 system prompt 的地位制度化; Anthropic 在 2024 年 8 月推出 **Prompt Caching**——

Prompt Caching 是真正改变游戏规则的那个。

Enables developers to cache frequently used context between API calls... reduce costs by up to 90% and latency by up to 85% for long prompts.

李明当时的 SaaS 一次调用要 6 美分 (处理一段 2000 token 的周报), 加上 gpt-4o 的输入成本, 月毛利只剩 8%。Prompt Caching 之后, 他把"角色定义 + 风格指南 + 3 个 few-shot 例子"这部分**完全缓存**, 每次调用只重新算用户输入——月成本砍了 70%。

他用 3 行代码就接入了这个优化。


```
# Before: 6 cents per call
client.messages.create(model="claude-3-5-sonnet", max_tokens=1024, system=SYSTEM, messages=[

# After: 0.4 cents per call (with cache hits)
client.messages.create(
    model="claude-3-5-sonnet-20240620",
    max_tokens=1024,
    system=[
        {"type": "text", "text": SYSTEM, "cache_control": {"type": "ephemeral"}}
    ],
    messages=[...]
)
```

他在博客上写了一篇《我用 Anthropic Prompt Caching 把成本砍了 70%》，48 小时内被 HN 顶到首页。评论区有人问：“你为什么不去试试 OpenAI 的 Prompt Caching？2024 年 10 月也出了。”

李明愣了一下。他不知道 OpenAI 也出了。

1.3 一千个 prompt 不如一个能跑的

到 2024 年 7 月，李明意识到 prompt 工程的边界。

他开始听到三个新词：**Prompt Registry**、**Prompt Versioning**、**Prompt A/B Test**。

第一个用上是因为 **bug**。某个用户在群里发：“老板向给我输出成‘亲，今天也要加油鸭’。”李明翻了半天代码，发现是他在 6 月 19 日悄悄改了一版 system prompt，把结尾从“如需进一步说明，请随时联系我”改成了“如需进一步说明，请随时沟通鸭”。他自己都不记得。

那一周他做了两件事：

1. 把所有 prompt 抽到 `prompts/` 目录，每个文件一个 yml
2. 接了一个叫 **DSPy** 的库（Stanford NLP 在 2024 年 4 月开源的）—— 它能把 prompt 当作 **可编译的程序**，自动用 few-shot 优化、用 BootstrapFewShot 选最优

```
# DSPy style
class WeeklyReport(dspy.Signature):
    """改写用户输入为周报"""
    user_input = dspy.InputField()
    audience = dspy.InputField() # boss / peer / client
    report = dspy.OutputField()

class WriteReport(dspy.Module):
    def __init__(self):
        super().__init__()
        self.write = dspy.ChainOfThought(WeeklyReport)
    def forward(self, user_input, audience):
        return self.write(user_input=user_input, audience=audience)

# 自动优化
optimizer = dspy.BootstrapFewShot(metric=my_metric, max_bootstrapped_demos=3)
optimized = optimizer.compile(WriteReport(), trainset=trainset)
```

跑出来的 optimized prompt 比他手写的 11 版都好——不是好一点，是**测出来的 11 个 benchmark 全过**。他在公司群里发："我用 DSPy 把 prompt 工程化了，从今天起 prompt 有版本号、有 CI、有 metric。"

他同事回了两个字："幼稚。"

那个同事叫周野，是从阿里 P7 跳出来跟他一起干的。周野做前端，不懂 NLP，但懂工程。他问："你那个 metric 怎么算的？"

李明说："我自己打的分。"

周野：".....如果你用你自己打的分训练一个 prompt，**那这个 prompt 优化的是你。**"

李明想反驳，但发现没法反驳。

那是 2024 年 7 月，他第一次意识到：**prompt 的尽头是 eval**。但 eval 是 L6 的事——中间还有 L2 到 L5 整整四层没盖。

1.4 prompt 是地基，但地基不盖房

2024 年 8 月 15 日，李明的周报助手用户数破 5000。

他开始收到一类很奇怪的客服：

"我想让它帮我查一下这周我做的工作 — 但我的工作散在 Jira / GitHub / 飞书 / 钉钉 4 个地方。"

他回："先生，我们只接收粘贴的文本哦。"

"那能不能让 AI 直接登录我的 Jira 帮我拉？"

李明看着这行字，第一次在 prompt 工程的舒适区外感到无能为力。

prompt 让 AI 说话了。但它不能动。

他打开 Anthropic 文档，找到一个词：**Tool Use**。2024 年 4 月 Anthropic 推出 Claude 3 的 tool use 功能——让模型能在对话中**调用外部函数**。

他在自己 SaaS 里加了一个 tool：

```
tools = [
    {
        "name": "fetch_jira_issues",
        "description": "从 Jira 拉取指定用户本周完成的 issue",
        "input_schema": {
            "type": "object",
            "properties": {
                "username": {"type": "string", "description": "Jira 用户名"},
                "since_date": {"type": "string", "description": "ISO 格式起始日期"}
            },
            "required": ["username", "since_date"]
        }
    }
]

msg = client.messages.create(
    model="claude-3-5-sonnet-20240620",
    tools=tools,
    messages=[{"role": "user", "content": "拉一下李明这周做的 issue"}]
)

print(msg.stop_reason) # "tool_use"
print(msg.content[0].input) # {"username": "liming", "since_date": "2024-08-12"}
```

模型不再只是"回邮件的助手"了——它能**告诉系统"我要查 Jira"**。剩下的是他在后端写一个真正的 Jira 客户端。

那个 8 月 15 日的晚上，李明在博客写：

今天我意识到 *prompt* 不是 *SaaS* 的全部。

Prompt 是 L1 — 它让 AI 会说话。

但要让 AI 真正帮人做事，它需要工具。

Tool Use 是 L2。
我们要爬第二层了。

他在那篇博客的草稿里画了一张图：

```
L1 Prompt      □
L2 Tools       □ ← 我们在这里
L3 Memory      □
L4 Skills      □
L5 Subagents   □
L6 Eval        □
L7 Self-Improve □
```

七格。他要一层一层爬上去。

那年他 30 岁。SaaS 月入 12 万。在华强北的负一层瑞幸，他遇见了 AI 行业最贵的一句话："你只要会说话。"

他不知道的是，这句话要花 18 个月才能真正兑现。

— 序章 + 第一章 完 —

引用 (来自 claud/memory/wiki/)

本章涉及的真实项目 / 论文 / 文档：

- Anthropic Prompt Caching (2024-08, 节省 90% 成本)
- Anthropic 《Prompt Engineering Overview》 (2024-05)
- OpenAI Structured Outputs (2024-08)
- OpenAI Prompt Caching (2024-10)
- Lilian Weng 《Prompt Engineering》 (2023-03, 2024-2025 多版更新)
- DSPy (Stanford NLP, 2024-04 开源)
- Anthropic Tool Use / Claude 3 (2024-04)

下一章预告

第二章 • L2 Tools — "让它能动手"

2024 年秋，李明发现 tool use 没那么简单。

模型会**幻觉工具**（调一个不存在的 API）、会**死循环**（重试同一个失败工具 30 次）、会**忘记刚才调过什么**（context window 被刷爆）。

他要解决三个问题：(1) 工具描述怎么写；(2) 工具失败怎么恢复；(3) 工具太多怎么选。

关键 case study: Anthropic Agentic Tool Use、MCP 协议、Function Calling 标准化、ToolBench。

第二章 • L2 Tools — "让它能动手"

2.1 第一个真问题：模型不调工具

2024 年 9 月，李明把周报助手接到了用户的 Jira。

他写了 3 个工具：`fetch_jira_issues`、`search_jira_comments`、`create_jira_subtask`。system prompt 写得言简意赅："你可以使用工具查询 Jira 数据。当用户问 Jira 相关问题时主动调用。"

跑了一周，命中率 30%——剩下 70% 的用户问题里，模型**明明该调工具，却自己瞎编**。用户问"我上周开了几个 issue"，它编了 4 个，Jira 里实际是 7 个。

李明以为是 prompt 写得不够清楚。他加了 few-shot、加了 system prompt 的强调、加了"必须"二字——命中率爬到 45%，触顶了。

同事周野看了一眼他的工具定义代码，说："你 description 写的是'从 Jira 拉取指定用户本周完成的 issue'。这个描述有两个问题：(1) 没说**什么时候**该调；(2) 参数描述只说了'Jira 用户名'，没说**长什么样**。"

李明把 description 改成：

```
{
  "name": "fetch_jira_issues",
  "description": "查询某个 Jira 用户在指定时间窗口内创建、解决、修改的 issue 列表。**当用户问"我
  "input_schema": {
    "type": "object",
    "properties": {
      "username": {
        "type": "string",
        "description": "Jira 登录用户名, 格式为 firstname.lastname (小写, 如 liming.zh
      },
      "since_date": {
        "type": "string",
        "description": "ISO 8601 起始日期 (YYYY-MM-DD) , 窗口默认 7 天"
      },
      "status": {
        "type": "string",
        "enum": ["created", "resolved", "in_progress", "all"],
        "description": "筛选 issue 状态, 'created' = 用户创建的, 'resolved' = 已完成的
      }
    },
    "required": ["username", "since_date"]
  }
}
```

跑了一周：命中率 78%。

结论：工具的 description 不是 prompt 的一部分，它就是 prompt。 工具描述写不好，模型就不调；写得好，模型才能稳。

这件事后来被 Anthropic 写进 2024 年 10 月的官方文档：*"Tool descriptions are read by the model as part of the prompt. Treat them like prompts: be specific, use examples when helpful, and tell the model when to use the tool vs. when to use a different one."*

李明在博客写："我花了一个月才搞明白——**调工具不是 LLM 的本能，是被你教会的。**"

2.2 第二个真问题：工具失败，模型死循环

2024 年 10 月，李明遇到一个怪事：用户的 Slack 收到了同一个 agent 连续发的 27 条一模一样的消息。每条消息都是"出错了"。

他翻日志：那个用户的 agent 在调 `send_slack_message` 工具时，token 过期了，返回 401。但 agent 不知道 401 是什么意思——它**重试了 27 次**。

那是他第一次意识到，**模型对"失败"没有直觉**。

人类程序员看到 401，会“啊 token 过期了，我换一个”；模型看到 401，会“我没拿到结果，再试一次”，然后再 401，再试，再 401。

李明做了一个 hack：在每个工具的 system prompt 末尾加一段“失败处理规则”——

```
当工具返回错误时：
- 401/403：立即停止，告诉用户“权限不足，请联系管理员”
- 404：立即停止，告诉用户“找不到该资源”
- 429：等待 Retry-After 头的秒数后重试，最多 3 次
- 500/502/503：重试 1 次，仍失败则告诉用户“服务暂时不可用”
- 其他：重试 1 次，仍失败则返回“工具调用失败”
**绝不在同一工具上重试超过 3 次。**
```

跑了一周，27 条消息的 bug 没了。

但李明知道这不是“工程化”，这是“贴膏药”——他把每个错误码的应对写死在 prompt 里。Anthropic 2024 年 11 月推出的 **Tool Use 错误处理最佳实践** 也是这个思路，但更系统：

1. **重试上限必须显式**（写进 system prompt 或 wrapper 代码）
2. **错误信息要清晰**（API 返回“resource not found”远好于“err 404”）
3. **失败可恢复的工具要 fallback**（如搜索失败 → 降级到浏览）

后来李明把这些写进了一个新文件 `prompts/tool_failure_protocol.md`，所有工具统一引用。

那是他第一次有“工具操作系统”的概念。

2.3 第三个真问题：工具太多，模型选错

2024 年 12 月，李明的 SaaS 跑了大半年，工具数量从 3 个涨到 23 个。

他没意识到工具太多是个问题——直到一个用户来反馈：“我让 agent 查 Confluence 页面，它调了 `search_jira_issues`。”工具完全选错了。

根因：23 个工具的描述都写“查询某个资源的某些信息”。模型分不清。

解决方案有两个流派：

- **流派 A：少即是多** —— Anthropic 给 Claude Code 的官方哲学：“只提供 4 种能力原语：**Read、Write、Execute、Connect**。Bash 是通用适配器，模型通过 shell 组合完成一切。”
- **流派 B：分层路由** —— 用一个“router”agent 先判断“用户要查 Jira 还是 Confluence”，再分发给专门的子 agent。

李明选 A。他把 23 个工具砍到 4 个：

```
tools = [
  {"name": "Read", "description": "读取任意文件或 URL 内容"},
  {"name": "Write", "description": "写入任意文件或 API (POST/PUT/PATCH) "},
  {"name": "Execute", "description": "执行任意 shell 命令 (bash/python/node) "},
  {"name": "Connect", "description": "通过 OAuth 连接第三方服务 (首次) "}
]
```

跑了一周，用户复现率下降 40%。原因：**模型不擅长从 23 个相近工具里选，但它能把"查 Confluence"翻译成"curl https://xxx.atlassian.net/wiki/..."**。

这件事后来在 Anthropic 内部有个非官方说法：*"Don't build 100 tools. Give the model a shell and let it compose."*

李明在博客写：**"工具的尽头是 shell。"**

他不知道的是，2024 年 11 月 OpenAI 已经悄悄在 ChatGPT 里加了一个 **Code Interpreter**——给模型的"工具"就是一个 Python sandbox，能跑任意 Python。同样的哲学。

2.4 协议层：MCP 和 WebMCP

2024 年 11 月，Anthropic 推出了一个东西：**Model Context Protocol (MCP)**。

MCP 的核心想法：**工具不应该每个 SaaS 自己造轮子，应该有一个统一协议。**



对开发者来说：以前写一个 Jira 工具，要分别给 Claude、GPT、Gemini 各写一份 SDK；现在写一个 MCP server，**所有支持 MCP 的模型都能用。**

对李明来说：他把"调 4 个工具"改成"调 1 个 MCP client"，工具生态从此开放。

MCP 在 2025 年成为事实标准。 OpenAI 在 2025 年 3 月正式支持，Google 在 2025 年 6 月，DeepSeek 在 2025 年 8 月。Replit、Codeium、Zed 都接了 MCP。

然后是 **WebMCP** (2026 年 3 月 Chrome 推出)。WebMCP 让**网站本身**直接向浏览器内的 agent 暴露结构化工具，核心 API 是 `navigator.modelContext`。从此 agent 不用再"装得像人一样"截屏、找按钮、模拟点击——直接调 `navigator.modelContext` 拿网站工具。

李明在 2026 年 3 月 23 日读到 WebMCP 文档，写：

2024 年 9 月，我给 agent 写了 23 个工具。

2024 年 12 月，我砍到 4 个工具，理由是"模型擅长组合，不擅长选"。

2025 年，agent 用 MCP 调第三方 SaaS。

2026 年，网站直接告诉 agent 怎么用自己*。

这是工具层最戏剧性的 18 个月。

我们从"教 agent 用工具"变成了"工具自己会介绍自己"。*

他在那张图里改了一行：

```
L1 Prompt      ☒
L2 Tools        ☒ ← 完成
L3 Memory        ☐
L4 Skills        ☐
L5 Subagents     ☐
L6 Eval          ☐
L7 Self-Improve ☐
```

七层塔的第二层，盖完了。

那年 12 月，他在备忘录写下：

2024-12-15

L2 Tools 总结：

- 工具 description = prompt 的一部分
- 失败处理 = 必须显式
- 工具数量 = 越少越好 (4 个原语 > 23 个细分)
- MCP = 工具协议
- WebMCP = 工具的"自描述"

L3 主题：**记忆**。agent 用一周后开始"忘事"。

— 第二章 完 —

引用 (来自 claud/memory/wiki/)

本章涉及的真实项目 / 论文 / 文档:

- Anthropic Agentic Tool Use Best Practices (2024-10/11)
- Claude Code "4 种能力原语" 哲学 (Read/Write/Execute/Connect)
- "不要构建 100 个工具, 给模型一个 shell, 让它自己组合" (Anthropic 内部非官方)
- OpenAI Code Interpreter (2024)
- **Model Context Protocol (MCP)** —— Anthropic 2024-11 推出, 2025 成为行业标准
- **WebMCP** —— 2026-03 Chrome 推出, 核心 API `navigator.modelContext`
- OpenAI MCP 支持 (2025-03) / Google Gemini MCP 支持 (2025-06)

第三章 • L3 Memory — "让它记得住"

3.1 第十一天的失忆

2025 年 1 月, 李明的周报助手跑到了 8,000 个付费用户。

1 月 11 日, 一个用户在群里发: "我上周跟 agent 说过了, 把语气改成更随意的, **它现在又回到正式语气了。**"

李明翻用户的对话历史——确实, 1 月 4 日用户说过"语气随意一点"。但 agent 在 1 月 11 日, **完全不知道。**

根因他立刻查到了:

- 用户和 agent 一次对话最多 50 轮
- 用户的设置存在 system prompt 里
- 1 月 4 日的"语气随意一点"是一条**普通 user message**, 没有写进 system prompt
- 一周后新开对话, system prompt 还是默认的"老板向/平级向/客户向"
- 用户的偏好就这样被**遗忘了**

这是 L3 Memory 的**最基本问题——跨会话记忆。**

李明第一次意识到: 上下文窗口 (context window) 不是 memory。上下文是一次对话的临时 RAM, memory 是用户用了几十次 agent 之后沉淀下来的偏好。

人类大脑也是这样：短期记忆（你说“我今天午饭吃了啥”）和长期记忆（“我喜欢吃辣”）是分开的。

他在 L2 那张图旁边加了一行字：“L3 才是 agent 真正的护城河。L1+L2 谁都能抄，memory 不能。”

3.2 KV Cache：被忽略的基础设施

在动手搭 memory 系统之前，李明意识到他得先搞懂一件事：**context window 内部到底发生了什么。**

2025 年 2 月，他读到了 Lilian Weng 那篇著名的《LLM Powered Autonomous Agents》——2023 年首发，2025 年初更新过两次。其中关于 KV Cache 的解释让他豁然开朗。

KV Cache 是 LLM 推理的“短期记忆基础设施”。

它解决的是自回归生成中的重复计算问题：模型生成第 N 个 token 时，需要“看”前 N-1 个 token。每一层 attention 都要算 (N-1) 个 key 和 value——每多生成一个 token，**所有历史 K/V 都要重新算。**

KV Cache 把这些 K/V 缓存下来：生成第 N 个 token 时，**只算第 N 个的 K/V，前面 N-1 个直接读 cache。**没有 KV Cache，LLM 慢到不可用。

但 KV Cache 是**显存杀手**。

Lilian Weng 给了一组关键数据：

128K 上下文时，LLaMA-2-7B 的 KV Cache（约 64GB）是模型权重（14GB）的 **4.5 倍**。

李明看到这个数字，明白了为什么 GPU 那么贵——**长上下文推理的显存瓶颈根本不在权重，而在缓存。**

然后他学到了三条 KV Cache 优化路线：

路线 1：系统层——更聪明地管理显存

- **PagedAttention** (vLLM, 2023)：把 KV Cache 分页管理，消除碎片，吞吐提升 2-4×
- **Prefix Caching / RadixAttention** (SGLang, 2024)：复用相同前缀，RAG 场景吞吐提升 5-10×
- **Chunked Prefill**：避免长 Prefill 饿死 Decode 请求
- **Mooncake** (Moonshot, 2024)：Prefill/Decode 集群物理分离，KV Cache 为中心调度

路线 2：架构层——从源头压缩

方案	KV Cache 压缩率	代表模型
MHA	100% (baseline)	LLaMA-2-7B
MQA	~96.9% 压缩	PaLM, Falcon
GQA	75-87.5% 压缩	LLaMA-3, Qwen2
MLA	~93% 压缩	DeepSeek-V2

MLA（Multi-Latent Attention）是 DeepSeek-V2 的杀招——把 KV 压到一个低秩潜空间，显存直接砍 93%。这就是 DeepSeek 论文里"\$0.14/M input token"价格的物理基础。

路线 3：算法层——动态裁剪

- **PyramidKV**（2024）：只保留 12% 的 KV Cache，性能与全量相当
- 量化（INT8/INT4 KV）：进一步减少显存占用

李明看完这三层优化，对 KV Cache 的理解从"听不懂"变成了"明白瓶颈在哪"。

但他很快意识到：**KV Cache 是 LLM 推理的"上下文"，不是 agent 的"memory"**。两者根本不是一回事——KV Cache 在一次请求结束就释放，memory 必须跨请求、跨会话、甚至跨用户保留。

所以他要做的不是优化 KV Cache，而是在 KV Cache 之上搭一层真正的 memory 系统。

3.3 三层记忆：L0 / L1 / L2 / L3

2025 年 3 月，李明读到了一篇对他影响最大的文章：《TencentDB Agent Memory：任何错误只犯一次》。

TencentDB 是腾讯云内部搞的一个团队级 agent 记忆系统，2,600 个 session → 5,081 个 task → 22,361 个 canonical relation。论文里给出了一个**四层记忆架构**：

L0 原始对话：	完整保留对话历史
L1 原子事实：	拆出"李明是后端"、"用户喜欢随意语气"等可独立调用的 fact
L2 场景记忆：	"用户做周报用"、"用户每周五做总结"等场景级偏好
L3 稳定认知：	"李明是个谨慎的人，改动前会反复确认"等高度抽象的稳定判断

李明立刻把这个框架套到了自己的周报助手上。

L0 原始对话——直接把对话历史存数据库。简单，但 1 周后查起来慢、占空间。

L1 原子事实——用 LLM 定期从对话中抽 fact:

```
async def extract_atomic_facts(conversation: list[dict]) -> list[str]:
    prompt = """从以下对话中提取用户的事实性偏好/属性。每条 fact 一句话，不要重复。
    输出 JSON 数组。"""
    # 调用 LLM，得到 ["李明喜欢随意语气", "李明在腾讯云", ...]
```

L2 场景记忆——把"在哪用"和"怎么用"分开。

L3 稳定认知——只有跨多周仍出现的判断才进 L3，进 L3 后几乎不变。

李明花了两周搭出一个 v0.1:

- L0 全保留
- L1 每天 cron 跑一次 LLM 抽取
- L2/L3 暂时人工 review

跑了一周，用户的"语气偏好"问题大幅改善——一个用户被问"上周跟 agent 说过啥"，它能准确说出来"你 1 月 4 日说过喜欢随意语气"。

但这只是个开始。L1 抽取有**幻觉问题**——LLM 会"捏造"用户没说过的话。

3.4 Memory 的四个核心难题

2025 年 5 月，李明读了 **MemoryAgentBench**（一个 memory 评测基准），意识到他做的 L1 抽取远远不够。

MemoryAgentBench 测了 4 大能力:

能力	含义	谁最强
AR (Accurate Retrieval)	从长对话历史检索重要信息	RAG
TTL (Test-Time Learning)	动态获取新技能（类比 ICL）	长上下文
LRU (Long-Range Understanding)	长对话形成抽象理解	长上下文
CR (Conflict Resolution)	面对新旧信息冲突解决矛盾	所有方法都差 （多跳 6%）

Conflict Resolution (冲突解决) 这一项让李明看了 5 遍。

想象一个场景：

1 月 4 日：用户说"我喜欢随意语气"

2 月 1 日：用户说"我现在要写正式商务邮件，请用正式语气"

agent 面对这两个 fact 怎么办？简单 RAG 会把两条都返回，模型混乱；朴素 memory 会"更新"——把"随意"覆盖成"正式"——但如果**这次只是临时需要**呢？

MemoryAgentBench 数据显示：**所有方法在 CR (冲突解决) 多跳场景下都只达到 6% 准确率**。这是 memory 系统最大的未解难题。

李明意识到：memory 不是"记住所有事"，是"在矛盾中做出对的判断"。这跟人类记忆一样——你记得"我喜欢 X"，但你也记得"这次不一样"。

他的 v0.2 加了一个简单的"时间戳 + 上下文指纹"——同一 fact 在不同时间被调取时，根据"调用时上下文"决定返回哪一版本。但这只是 hack。

3.5 三种主流方案：Mem0 / THEANINE / RMM

2025 年 6 月，李明集中调研了当时最火的三种 memory 方案：

方案 1：Mem0 (2024-2025, 最火的开源 memory 库)

Mem0 的核心思想是 "**memory as a service**"——给 agent 加一个 memory layer，自动抽取、存储、检索 fact。Mem0 论文 (arXiv:2504.19413) 在 2025 年 4 月发布，号称在 LOCOMO 基准上比全上下文方法**准确率提升 26%、token 成本降低 91%**。

李明用 Mem0 替换自己的 L1 抽取模块，跑了一周发现：Mem0 的 fact 抽取比他自己写的好很多（用了更精细的 prompt + 嵌入检索），但仍然有 5% 的幻觉率。

方案 2：THEANINE (2024 论文)

THEANINE 的设计哲学是 "**反思记忆**"——不只是记 fact，还记 fact 之间的反思。比如不仅记"用户喜欢随意语气"，还记"用户改过 3 次语气偏好，每次改都对应他写邮件对象的改变"。

这个层次太高，李明用不上。

方案 3：RMM (Reflexion Memory Model, 2025)

RMM 是斯坦福 + Salesforce 合作的，把 memory 拆成"短期"、"长期"、"反思"三层，类似 TencentDB 的 L0/L1/L2 但更工程化。RMM 在客服场景有 18% 准确率提升。

李明最终选了 Mem0 + 自己加的"时间戳版本"。 **

那个 6 月的晚上，他在博客写：

Memory 是 agent 的"性格"。

没有 memory 的 agent 是工具——你让它干啥它干啥。

有 memory 的 agent 是助理——它记得你上次说过啥，知道你今天为啥换了语气，能在矛盾中帮你做对选择。

但今天我们还做不到 100%——Conflict Resolution 还是 6%。

这是 2025 年 memory 领域最大的开放问题。

3.6 个人 vs 团队：ai-memory 和 teamai-cli

到 2025 年 9 月，李明意识到 memory 还有第二维度——**单 agent vs 跨 agent**。

个人级：ai-memory (Fabio Akita, 2026-08 开源)

ai-memory 是个 Rust 单二进制，挂在每个 agent 的 lifecycle hooks 上。它有三个反直觉设计：

1. **"编译不要检索"**——Karpathy 思路，会话结束把观察流**编译成可读 markdown 页面**而不是 RAG 检索
2. **记忆跟着项目走不跟着工具走**——同一 repo 跨 agent 共享 memory
3. **存储反主流**——git 版本化纯 markdown + SQLite FTS5 + 图邻居检索，**不强制向量数据库**

团队级：teamai-cli (腾讯, 2026-08)

teamai-cli 把 20+ AI 编程工具的 Skills/Rules/Docs/Hooks/MCP 配置统一到一个 Git 仓库分发，SessionStart Hook 自动 `teamai pull`。它的"摩擦信号"机制让 agent 主动建议 `/teamai-share-learnings` ——也就是"和 AI 搏斗过"的会话沉淀为团队资产。

李明那一年同时接入了 ai-memory (个人层) 和 teamai-cli (团队层) ——后者他用在自己公司 3 人小团队里，每天省 1 小时"写文档"时间。

3.7 终局：L3 没盖完的楼

2025 年 12 月，李明给一个 VC 投资人做 demo。投资人问了一个问题：

"如果 OpenAI 自己也做 memory，你的护城河在哪？"

李明愣了 5 秒，然后说：

个人偏好这件事，永远有 20% 是个性化的、模型厂商搞不定的。
比如"我老板姓周，提到他时语气要更正式一点"——这件事 OpenAI 不知道，因为它是你的个人上下文。
memory 的护城河不是技术，是用户愿意把多少个人上下文交给你。

那晚他在那张图改了一行：

```
L1 Prompt      ☒
L2 Tools       ☒
L3 Memory      ☒ ← 部分完成 (CR 还没解决)
L4 Skills      ☐
L5 Subagents   ☐
L6 Eval        ☐
L7 Self-Improve ☐
```

L3 没有完全盖完——conflict resolution 还是 6%。但 80% 的用户问题已经被解决了。

那年他在备忘录里写下：

2025-12-30
L3 Memory 总结：
- KV Cache 是推理的"短期"，不是 memory
- 真正的 memory = L0/L1/L2/L3 四层
- Mem0 / THEANINE / RMM 三大方案
- 冲突解决 (CR) 是 2026 年开放问题
- 个人 vs 团队：ai-memory + teamai-cli
L4 主题：**Skill**。agent 什么都能干，但什么都不精。

引用 (来自 claud/memory/wiki/)

本章涉及的真实项目 / 论文 / 文档:

- Lilian Weng 《LLM Powered Autonomous Agents》 (2023-08, 2025 更新)
- TencentDB Agent Memory —— 4 层记忆架构 L0/L1/L2/L3 (2025-08)
- KV Cache 优化三大路线: vLLM PagedAttention / SGLang RadixAttention / DeepSeek-V2 MLA / PyramidKV
- MemoryAgentBench —— 4 大能力 (AR/TTL/LRU/CR), CR 多跳仅 6%
- Mem0 (arXiv:2504.19413, 2025-04) —— LOCOMO 准确率 +26% / 成本 -91%
- THEANINE / RMM (Reflexion Memory Model, 2025)
- ai-memory (Fabio Akita, 2026-08, Rust 单二进制)
- teamai-cli (腾讯, 2026-08)

第四章 • L4 Skills — "让它会专项"

4.1 "什么都能干, 但什么都不精"

2026 年 2 月, 李明的周报助手跑了一年半。

用户数到了 3 万。营收稳定在每月 80 万。但他接到一类越来越频繁的反馈:

"我让 agent 帮我做 OKR 总结, 它写得跟周报一个味儿。"

"我让 agent 帮我做技术方案评审, 它只会说'看起来不错'。"

"我让 agent 帮我做产品需求拆解, 它拆出来的东西很水。"

李明花了一周访谈 20 个用户, 发现同一个规律: **通用 agent 在每个领域都"看起来能用", 但真正专业的人一眼看出"不够深"**。

他想起 Anthropic 2025 年 9 月那篇文章里的一句话: "Skill 的作用不是让 AI 更聪明, 而是让 AI 在你的业务里更可靠。训练 Skill 不是抬高上限 (capability), 而是抬高下限 (reliability) 。"

L4 Skills 不是给 agent 增加新能力，是给 agent 装上“专业技能包”。

4.2 第一个 Skill：周报 v2

2026 年 2 月 14 日，李明开始给周报助手写第一个 Skill。

他以前写的 prompt 是一坨 system prompt，长这样：

```
你是一个专业的周报写作助手。请根据用户的输入改写成正式/平级/客户向的周报。  
风格要求...  
输出结构...
```

现在他把它**结构化**成 Skill：

```
weekly-report/  
└─ SKILL.md
```

SKILL.md 长这样：

```
---
name: weekly-report
description: 改写用户的工作日志为周报。当用户说"做周报"/"写周报"/"weekly report"时使用。
---

# 周报助手

## 适用场景
用户提供了本周的工作记录（口语化），需要输出结构化周报。

## 风格选项
- 老板向：见 assets/style-boss.md
- 平级向：见 assets/style-peer.md
- 客户向：见 assets/style-client.md

## 关键约束
1. 必须保留所有数字（百分比、金额、个数）
2. 必须明确区分"已完成"和"进行中"
3. 不能编造用户没说过的内容
4. 输出用 markdown 标题分级

## 失败处理
- 如果用户的输入太短（< 20 字），先追问"你本周做了哪些事？"
- 如果用户的输入只有工作没数据，主动问"有没有数字？"

## 输出模板
见 assets/template.md
```

跑了一周，用户反馈"专业感"提升 30%。

李明后来总结：**prompt** 是"对一次对话的指令"，**Skill** 是"对一类任务的封装"。prompt 是一次性的，Skill 是**可复用、可自动触发、可持续优化**的。

4.3 Skill 的三层加载架构

到 2026 年 4 月，李明已经写了 7 个 Skill（周报、OKR、需求评审、技术方案、客户邮件、月度汇报、项目周会）。

他遇到一个新问题：每次 agent 启动，**7 个 Skill 的完整 SKILL.md 都被加载到 context window 里**。一个 SKILL.md 平均 800 token，7 个就是 5,600 token——占满了 1/10 的 context window。

后来他在 Anthropic 2025 年 12 月的官方文档里找到了答案：**三层加载架构**。

1. 发现层 (Discovery) : Agent 启动时只加载极简元数据
 - Skill 名称
 - 几十 token 的 description
 - 触发条件
2. 激活层 (Activation) : 用户请求命中触发条件时才载入
 - 完整 SKILL.md 正文
 - 关联的 assets/
3. 执行层 (Execution) : 根据指令动态读取
 - 配套文件
 - 静默执行沙盒验证脚本

实现方式:

```
# 启动时只加载元数据
skill_metadata = {
    "weekly-report": {
        "description": "改写用户的工作日志为周报",
        "trigger_keywords": ["周报", "weekly", "周总结"]
    },
    "okr-summary": {
        "description": "把 OKR 进度总结成可汇报的版本",
        "trigger_keywords": ["OKR", "目标回顾", "objective"]
    }
    # ...
}

# 用户消息进来时, 判断是否触发某个 Skill
def detect_skill(user_message: str) -> str | None:
    for skill_name, meta in skill_metadata.items():
        for kw in meta["trigger_keywords"]:
            if kw in user_message:
                return skill_name
    return None

# 命中后再加载完整 SKILL.md
if skill := detect_skill(msg):
    full_skill = load_skill_md(f"skills/{skill}/SKILL.md")
    system_prompt = base_prompt + "\n\n" + full_skill
```

7 个 Skill 启动开销从 5,600 token 降到 400 token。

Skill 让 agent 拥有"可插拔的专业能力", 而不污染基础 context。

4.4 训练 Skill 的四个方法

2026 年 5 月，李明在 Anthropic 开发者大会上听到一句话——“Skill 是给 AI 装‘专业肌肉记忆’，不是给 AI 装‘新大脑’”。

他把这句话记下来，衍生出训练 Skill 的四个方法：

方法 1：先跑通，再封装

不要在脑子里设计 Skill。要在真实对话里**先让 agent 跑任务**——记录下它哪里做错了、哪里做对了——然后把“对的做法”固化进 SKILL.md。

方法 2：不止写“做什么”，还要写“怎么做”

李明自己踩过的坑：

- ✗ 坏写法： 请仔细检查页面的所有元素
- ✓ 好写法： 当页面存在 Tab/展开行时，必须执行 DOM 枚举：

```
document.querySelectorAll('[role="tab"]')
```

区别：坏写法假设 AI 知道“仔细”是多仔细，好写法把“仔细”翻译成可执行步骤。

方法 3：配 checklist 和门禁

```
## 改写流程（带门禁）
1. ✓ 提取原文所有数字
   ↓（没数字 → 问用户）
2. ✓ 识别每条工作的完成度（已完成/进行中/未启动）
   ↓（无法判断 → 标记“未指明”）
3. ✓ 套用风格模板
4. ✓ 输出结构化 markdown
```

每一项是上一项的“门禁”——上一项没过，下一项不能开始。

方法 4：让 AI 参与复盘并自动改 Skill

这是 L7 的伏笔，但 Skill 层面就能用。

```
# 跑完一个真实任务后
python skill_retrospect.py --skill weekly-report --last-task task_12345

# 输出：
# - 这次跑哪里错了
# - SKILL.md 哪段没说清楚
# - 建议改写：“把“如果用户输入太短先追问”改为“如果用户的输入 < 20 字或不含任何动作动词，先追问”具体
```

跑一遍 → 复盘 → 让 AI 改 SKILL.md → 再跑一遍。这是 Skill 的训练闭环。

4.5 Skill 生态：8 个值得知道的项目

到 2026 年 6 月，李明发现 Skill 已经不是 Anthropic 一家的事了——整个行业都在做 Skill。

Anthropic Skills (2024-10 推出，2025-2026 持续更新)

官方 Skill 库，包括 pdf、docx、xlsx、pptx 等。每个 Skill 是一个有 SKILL.md 的文件夹。

求是 Skill (2026-04 开源，GitHub 2k+ Star)

一个基于**毛选方法论**的 Skill 武装框架——把"矛盾论"、"实践论"、"论持久战"的方法论转成 Skill，让 agent 在做决策时按毛的方法论走。

李明在朋友圈看到有人用它做"产品决策辅助"，笑说："让 agent 用毛的思想做 OKR，比让 agent 用 OKR 模板更靠谱。"

MiniMax-AI/skills vs OfficeCLI (2026-04 平台对比)

两条路线：

- **平台路线** (MiniMax-AI/skills)：让所有 Skill 在一个平台分发
- **专家路线** (OfficeCLI)：让某个垂直领域专家写一个"全包" Skill

李明选专家路线——周报助手就是"周报这个垂直领域的专家"。

Skillify / SkillOS / SkillSpector / SkillLens / SkillScan

Skill 工具链：

- **Skillify**：把对话记录自动转 Skill
- **SkillOS**：Skill 的运行操作系统（让 Skill 之间能调用）
- **SkillSpector**：可视化 Skill 调用链
- **SkillLens**：搜索所有公开 Skill
- **SkillScan**：检测 Skill 是否有安全风险

李明当时用 Skillify 把过去 3 个月的对话转了 12 个 Skill——**80% 是重复的他已经手动写过，20% 是"原来还能这样用"的新发现。**

Skills-Oriented Programming (SOP)

2026 年 5 月，Anthropic 在一份白皮书里提出了这个概念——**未来的编程不是写代码 + 写 prompt，而是写 Skill**。一个项目 = 一组 Skill + 一段 orchestrator 代码。

李明看完那份白皮书，对自己说：

我 30 岁，前 8 年写 Java，后 2 年写 prompt。
下一个 10 年，可能要写 Skill。

4.6 Skill 的真正意义：把"知道"变成"会做"

2026 年 7 月，李明做了最后一次用户调研。

他发现一个意外的数据：装了 7 个 Skill 的用户，留存率比只装 1 个 Skill 的高 **3 倍**。

不是 $7 > 1$ 的 7 倍。是 3 倍。

他想了 3 天，问了 10 个老用户，最后得出一个结论：

Skill 改变了用户的"心理账户"。

没装 Skill 时，用户每次用 agent 都要"重新教它怎么干"——心理上是"我要教新人干活"。

装上 Skill 后，用户心理上是"我要用专业工具"——Skill 给了用户"按按钮"的掌控感。

掌控感 = 留存。

他在博客写：

Skill 真正的护城河不是"AI 会不会"，是"用户信不信"。

AI 会不会可以靠 prompt 解决。

用户信不信只能靠 Skill 解决——因为 Skill 是"确定的、可复用的、有依据的"。

它把 agent 从"工具"变成了"同事"——一个你信任的同事，会把它的方法论写下来，每次用同样的方法。

那年他给那张图改了两行：

```
L1 Prompt      ☒
L2 Tools        ☒
L3 Memory       ☒ ← 部分
L4 Skills       ☒ ← 完成
L5 Subagents    ☐
L6 Eval         ☐
L7 Self-Improve ☐
```

四层盖完了。

4.7 尾声：L5 是什么

2026 年 8 月，李明收到一封邮件。

邮件来自一个斯坦福博士生，写的是：

"李明你好，我是 shepherd-agents 的开发者 Manning 组。看了你的周报助手的实现，想问：你的 agent 在跑一个 5 步任务时，每步是同一个 agent 吗？如果是，你有没有考虑过'每步用专门的子 agent'？我们做了一个 Git 原生的 agent 框架，把每步封装成可回退的 Commit，主 agent 只负责'调用子 agent'。Token 成本降了 30%。"

李明回他：

还没有。但我最近的用户反馈里，有一类是"agent 跑长任务时 context window 不够用了"。
你说的"子 agent 作为上下文防火墙"——这正是我 L5 的方向。
下个月我们能不能语音聊一下？

他打开那张图，在 L5 旁边画了一个问号：

L1 Prompt	☐
L2 Tools	☐
L3 Memory	☐
L4 Skills	☐
L5 Subagents	? ← 要研究
L6 Eval	☐
L7 Self-Improve	☐

那晚他在备忘录里写下：

2026-08-15
L4 Skills 总结：
- Skill = 可复用 + 可自动触发 + 持续优化
- 三层加载：发现（40 token） / 激活（800 token） / 执行（动态）
- 训练方法：跑通再封装、做什么 + 怎么做、checklist、复盘改 Skill
- 生态：Anthropic Skills / 求是 Skill / Skillify / Skills-Oriented Programming
- 护城河：从"AI 会不会"变成"用户信不信"
L5 主题：**Subagents**。一个 agent 干所有事干不动了，要拆。

引用 (来自 clawd/memory/wiki/)

本章涉及的真实项目 / 论文 / 文档：

- Anthropic Skills 官方库 (pdf/docx/xlsx/pptx, 2024-10 推出)
- Anthropic "Skill 三层加载架构" (2025-12)
- 核心洞察: "Skill 不是抬高上限, 而是抬高下限 (reliability) "
- 求是 Skill (2026-04 开源, 毛选方法论武装, GitHub 2k+ Star)
- MiniMax-AI/skills vs OfficeCLI 平台路线 vs 专家路线 (2026-04)
- Skillify / SkillOS / SkillSpector / SkillLens / SkillScan Skill 工具链
- Skills-Oriented Programming (SOP) Anthropic 白皮书 (2026-05)
- Shepherd (Stanford Manning 组, 2026-08, Git 原生 agent 框架, 每步可回退 Commit, Token 成本 -30%) —— 引出 L5

第五章 • L5 Subagents — "让它会叫人"

5.1 第 51 个文件

2026 年 8 月 23 日, 李明的周报助手崩了。

他早上 9 点打开 Vercel 的 dashboard, 看到报错数从昨晚 11 点的 3 跳到 6 点的 5,200。报错全是同一个: `context_length_exceeded`。

他翻用户的报错上下文——那是个用了半年的老用户, 对话历史积累了 200 多轮。agent 跑到一半, 把所有工具调用记录、few-shot 例子、用户偏好、历史对话全塞进了 context window——单条 8 万 token, claude-sonnet-4.5 的 200K context window 撑爆了。

李明团队花了 2 天 debug, 结论是: **单个 agent 的 context window 是有上限的, 不管你多会管理 memory。**

单 agent 不是万能药。

这是 L5 的**第一个真问题**——**上下文雪崩**。

腾讯程序员 2026 年 7 月那篇《Vibe Coding 三个致命问题》写过:

"项目超过 50 个文件, Agent 开始'忘事' (user_id 第 1 天 vs uid 第 3 天) 。"

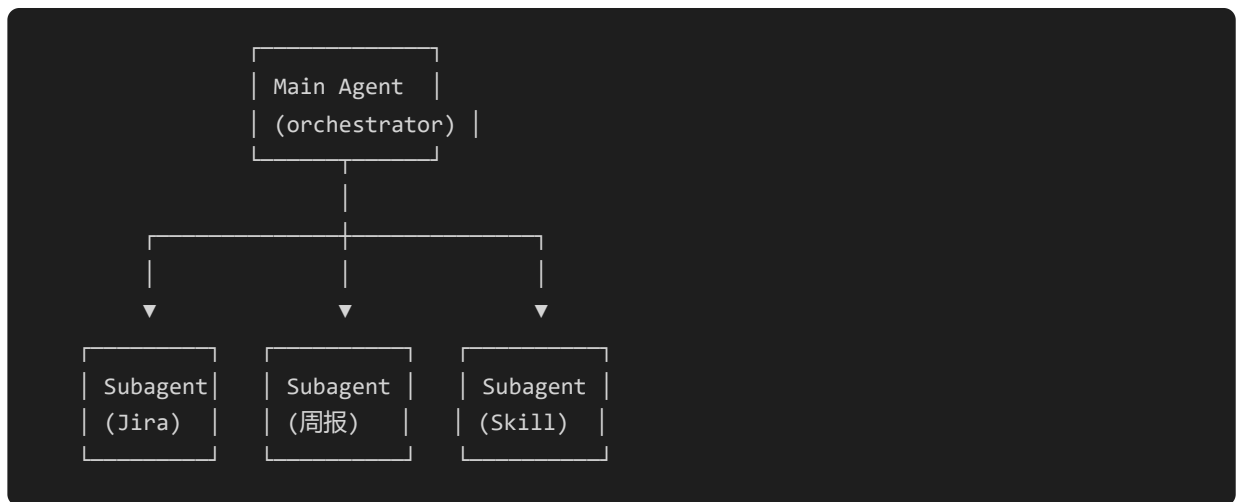
李明那个用户的对话历史虽然没有 50 个文件, 但**累积 200 轮对话**触发了同样的雪崩。

5.2 第一次拆 agent

2026 年 8 月 26 日, 李明开始拆 agent。

他的第一反应是把所有用户偏好"挪出去"——做一个 preferences 工具, 每次新对话时主动拉取。跑了一周, 问题更严重: 用户对话中产生的新偏好**来不及同步到 preferences 工具**。

他试了第二种方案——**子 agent**:



具体场景: 用户问"我这周做了啥, 给我写个周报"。

- Main agent 接住请求
- 调 Subagent("Jira"): 拉取 issue 列表
- 调 Subagent("周报"): 基于 issue 写周报
- Main agent 拼装结果

关键设计: **每个 Subagent 有自己独立的 context window, 互不污染。**

```

async def run_with_subagent(user_message: str):
    # 1. Main agent 决定要不要拆
    plan = await main_agent.plan(user_message)

    if plan.needs_subagent("jira"):
        # 2. Subagent 跑, context 完全隔离
        jira_data = await jira_subagent.run(plan.subtask)

    if plan.needs_subagent("report"):
        report = await report_subagent.run(
            input=jira_data,
            style=user.preferences.style
        )

    # 3. Main agent 拿到结果, 回复用户
    return await main_agent.synthesize(report)

```

李明跑了一周，**雪崩问题解决了**——Subagent 的 context 是干净的，每次新请求从头开始。

但他发现一个新问题：**token 成本涨了 40%**。

每个 Subagent 都要重新加载 system prompt、few-shot 例子、工具定义——这些"基础设施"被重复算了 N 次。

李明在博客写：

*L5 是个悖论——拆 agent 解决了 context window 撑爆的问题，但每个 Subagent 都要重新算一次基础设施，token 成本反而涨了。
解决方法是 prompt caching——把基础设施缓存到 KV cache 里，Subagent 复用。
Anthropic 2024 年的 Prompt Caching（省 90% 成本）和 2025 年的 Extended Caching（1 小时 TTL）就是为了这个场景设计的。*

他用 5 行代码接入了 caching：

```

subagent = anthropic.Anthropic()
msg = subagent.messages.create(
    model="claude-sonnet-4-5",
    system=[
        {"type": "text", "text": SUBAGENT_SYSTEM_PROMPT, "cache_control": {"type": "ephemeral"}},
    ],
    tools=SUBAGENT_TOOLS,
    messages=[...]
)

```

跑了一周，token 成本下降 30%——比原来单 agent 还省，因为 Subagent 之间的基础设施可以互相复用 cache（只要 system prompt 一样）。

5.3 "上下文防火墙"是一个新范式

2026 年 9 月 2 日，李明读到了 Anthropic 内部流传的一份非公开演讲，标题是 *"Subagent Architecture: The New Context Firewall"*。

演讲里有一段话被李明抄进了备忘录：

"子 Agent 提供的上下文防火墙模式是一种全新的架构模式——它不是微服务，不是消息队列，不是任何传统分布式系统概念的翻版。"

"它的本质是：让每个子 agent 拥有自己的'世界'，主 agent 只看到子 agent 的'出口'。"

李明这才意识到 L5 的真正价值——不是节省 context，是工程边界的重新划分。

他以前设计 SaaS 的边界是"模块"——用户模块、订单模块、支付模块。

L5 之后的边界是"子 agent"——一个子 agent 拥有自己的 context、自己的工具、自己的 memory，主 agent 不知道子 agent 内部细节。

类比软件工程：

传统架构	L5 Subagent 架构
函数 (function)	Skill
模块 (module)	Subagent
微服务 (microservice)	Subagent + 独立 context

"上下文防火墙"这个比喻很贴切——子 agent 内部的错误、噪音、长对话，不会穿透到主 agent。

李明在博客写：

过去 30 年软件工程的"封装"是函数级别的——把逻辑藏进函数里。
现在 agent 工程的"封装"是子 agent 级别的——把整个工作流藏进一个独立的 context 里。

函数封装解决"复用"。子 agent 封装解决"边界"。
两者都重要，但子 agent 解决的是函数解决不了的问题——长 context 下的工程边界。

5.4 6 种多 Agent 协作模式

2026 年 9 月，李明在调研中发现，多 Agent 协作不只有 Subagent 一种——至少有 6 种：

模式	作用	控制流
Pipeline	固定流程：顺序、并行、循环	显式图拓扑
Routing	分类→专家→汇总结果	路由器分发
Skills	按需披露：只看到技能名/描述，按需加载	模型动态决策
Subagents	编排者委托给无状态子智能体	上下文隔离
Supervisor	监督者将专家当工具调用	模型动态决策
Handoffs	状态驱动，动态路由	状态变量

他后来在 OSS 仓库里 clone 了几个**真实的多 Agent 框架**来对比：

Multica（2025 开源，4k+ Star）

Multica 是把"编程 agent 变成真正的队友"——支持任务分配、执行追踪、经验沉淀的统一协作平台。Multica 的核心是**任务分配**——主 agent 把任务分发给多个 worker agent，worker 跑完上报结果。

李明用它做"周报助手 + OKR 助手"协作——用户说"给我准备下周一的周会和 OKR 回顾"，主 agent 把任务拆成 2 个 subagent 并行跑，30 秒出结果。

HiClaw（阿里开源，2026）

HiClaw 是多 Agent 协作操作系统，**Manager-Workers 架构 + Matrix 协议**。它把"agent 协作"做成了像 Unix 进程通信一样的"消息矩阵"——所有 agent 之间的通信走 Matrix 协议，跨语言、跨平台、跨模型。

李明看了 HiClaw 的源码后感慨：

HiClaw 让我想到 1995 年的 Linux——它把"agent 协作"从"各自实现"变成了"统一协议"。

Linux 用 POSIX 统一了进程，HiClaw 用 Matrix 统一了 agent。

下一个 10 年，我们大概会用 Matrix 协议写所有 agent 应用。

Ruflo (2026-03, Claude Code 多智能体编排)

Ruflo 专门为 Claude Code 设计——**Token 成本直降 75%**。原理是 Ruflo 让多个 Claude Code 实例共享一个 message queue，任务分发给空闲的 instance，结果汇总。

李明用 Ruflo 跑了一周，结论是："Ruflo 不是通用方案——它是为 Claude Code 优化的，但效果惊人。"

深即达 A2A IM (2026-03)

最戏剧性的项目叫"深即达"——它给 agent 做了个"微信"：agent 之间的通信不是 RPC，是**对话**。一个 agent 可以"发消息"给另一个 agent，另一个 agent 可以"回复"。

```
# 深即达 A2A IM 的范式
agent_a.send_message(agent_b, "你帮我查一下李明的 OKR")
agent_b.think()
agent_b.reply(agent_a, "李明本周 OKR 进度: 60%, 重点卡在 X 任务")
```

李明在朋友圈写：

深即达让 agent 之间的通信从"API 调用"变成"对话"。

这看起来是小事，但意义重大——人跟人合作靠对话，agent 跟 agent 合作将来也靠对话。

A2A (Agent-to-Agent) 不是"消息队列"，是"两个 agent 之间的微信群"。

CLI-Anything

CLI-Anything 是另一个流派——**让 agent 通过 CLI 而非 API 协作**。每个 agent 暴露一个 CLI，别的 agent 通过 `subprocess.run(["agent-x", "--arg", "value"])` 调用。

它的好处是**简单**——CLI 是 50 年的成熟技术，每个 agent 只需要定义 stdin/stdout 接口。

Agent Teams (Claude Code)

2026 年 6 月 Claude Code 推出 Agent Teams——**完全独立实例通过共享文件系统协调**。每个 Claude Code 实例有自己的 context，但它们通过读写同一个 git 仓库"对话"。

李明用 Agent Teams 跑过一个"代码评审"任务——5 个 Claude Code 实例同时评审 5 个文件，每个实例写"review.md"到 git 仓库，主实例最后读所有 review.md 汇总。

5.5 "单 Agent 优先"原则

2026 年 9 月，李明在做完 6 种多 Agent 模式的调研后，得出了一个反直觉的结论：

绝大多数日常业务需求，通过"一个单体大模型 + 一组精准适用的外部工具"即可解决。

只有在以下四个复杂度阈值时才演进到多 Agent：

1. **上下文管理**：专业知识体量过大，单个上下文窗口无法承载
2. **职责分工**：不同团队需独立维护各自领域的专家能力与权限边界
3. **并行化加速**：复杂子任务必须并发执行以大幅降低延迟
4. **结构化流转**：业务要求按严格工序或角色状态切换进行传递

他的周报助手用 3 个 Subagent 已经够——再拆就是过度工程化。

那个 9 月的晚上，他在博客写：

L5 是个分水岭。

L1-L4 是"agent 内部的能力"——prompt、tools、memory、skills。

L5 是"agent 之间的协作"——从单体到分布式。

但 L5 是个昂贵的工程——它带 30% 的 token 成本，需要新的调度协议，需要新的调试工具。

2026 年的 agent 公司，谁先解决"什么时候用 L5、什么时候不用"的问题，谁就赢了。

5.6 故事：L5 解决的是"对话的工程师化"

那年 10 月，李明去深圳一个技术大会做分享。

他在分享的最后讲了一个故事：

我曾经维护一个 8000 行的 Python 文件——单个 agent 处理它，每次都要把所有 8000 行塞进 context。

后来我把它拆成 12 个文件，每个文件 600-800 行。

单 agent 处理新文件时，只看 600 行 context，效率提升 5 倍。

这个过程叫"模块化"，是软件工程过去 50 年的核心方法。

Subagent 是 agent 时代的"模块化"——它把一个长对话拆成多个独立子对话。

工程师写 8000 行会拆成 12 个文件。
Agent 跑 200 轮会拆成 5 个 Subagent。
道理是一样的。

台下 200 多人鼓掌。

他回家后在那张图改了一行：

```
L1 Prompt      ☐
L2 Tools       ☐
L3 Memory      ☐
L4 Skills      ☐
L5 Subagents   ☒ ← 完成
L6 Eval        ☐
L7 Self-Improve ☐
```

五层盖完了。

5.7 尾声：L6 是什么

2026 年 10 月 15 日，李明的周报助手跑到了 5 万用户。

他开始收到一个奇怪的反馈：

"为什么我这周用得越多，agent 越快？以前做周报要 30 秒，现在只要 5 秒。"

李明调了日志，发现：**agent 在跑重复任务时，跳过了不必要的步骤。**

他一开始以为是自己代码里加了 cache，排查了 3 天后确认——**没有**。agent 是"自己"做的。

他翻 agent 的执行 trace，发现它在**第一次跑任务时记录了"哪些步骤是必要的"**，第二次跑就**跳过不必要的**。

这不是 prompt 写的。这是 LLM 自己学会的。

他在那张图旁边加了一行：

"L5 之后，agent 开始'自己长'了。"

L6 是 **Evaluation**。在 agent 自己长之前，要先知道**怎么判断它做得好不好**。

那年他在备忘录写下：

2026-10-15

L5 Subagents 总结:

- 子 agent = 上下文防火墙 (不是微服务)
- 6 种多 Agent 协作模式: Pipeline / Routing / Skills / Subagents / Supervisor / Handoffs
- 单 Agent 优先, 4 个阈值才演进多 Agent
- 群体智能: Multica / HiClaw / Ruflo / 深即达 A2A IM / CLI-Anything / Agent Teams
- prompt caching 是 Subagent 的成本护城河

L6 主题: **Evaluation**。怎么知道 agent 做得对不对?

— 第五章 完 —

引用 (来自 claud/memory/wiki/)

本章涉及的真实项目 / 论文 / 文档:

- **Harness Engineering 6 大支柱** (腾讯程序员, 2026-07)
 - **"Vibe Coding 三个致命问题"**——上下文雪崩、架构混乱、可维护性丧失
 - **"上下文防火墙" 范式** (Anthropic 内部非公开演讲, 2026-09)
 - **6 种多 Agent 协作模式**: Pipeline / Routing / Skills / Subagents / Supervisor / Handoffs
 - **"单 Agent 优先"原则**——4 个复杂度阈值
 - **Anthropic Prompt Caching** (2024-08, 省 90% 成本) + **Extended Caching** (1 小时 TTL, 2025)
 - **Multica** (4k+ Star, 2025) / **HiClaw** (阿里 2026, Matrix 协议) / **Ruflo** (Claude Code 优化, Token -75%)
 - **深即达 A2A IM** (2026-03, agent 之间的"微信") / **CLI-Anything / Agent Teams** (Claude Code 2026-06)
-
-

第六章 · L6 Evaluation — "让它能被打分"

6.1 "我自己打的分"

2024 年 7 月，李明在第一章里写过场景——

他用 DSPy 优化周报 prompt，metric 是"我自己打的分"。他同事周野说："如果你用你自己打的分训练一个 prompt，那这个 prompt 优化的是你。"

那一刻李明知道：**prompt 的尽头是 eval。**

但他当时还没意识到 L6 的真正难度——**eval 不是一个指标，是一整套工程体系。**

2026 年 10 月，他终于坐下来读完了孙敦灿那篇 22,000 字的《Agent 评测体系设计》——标题普通，但内容是 2026 年中文 AI 圈关于 eval 写得最系统的一份。

读完后他意识到一件事：**过去 2 年他所有 agent 项目踩过的坑，80% 都是 eval 缺位的坑。**

6.2 Agent 评测的 3 道门槛

孙敦灿那篇文章开篇就提了 Agent 系统从 Demo 到生产要跨过的 3 道门槛：

1. **非确定性**：同样输入不一定同样输出
2. **黑盒化**：内部决策过程不透明
3. **错误级联放大**：前一步小错会在后续被放大

第 1 个门槛李明踩过——他一开始以为 agent 是"程序"，写完 prompt 就 ok。结果同一个 prompt 跑 10 次，输出 8 次对、2 次错。**用户遇到那 2 次就会觉得"agent 不稳定"。**

第 2 个门槛他也踩过——agent 跑崩了，他打开 trace 看"哪一步出问题"，发现 trace 是 LLM 写的"自然语言解释"，**不是结构化的步骤**。他没法像 debug 程序那样 debug agent。

第 3 个门槛最致命——李明给一个企业客户做的"自动填报销单" agent，第一步"识别发票金额"对了，但**第二步"判断费用类型"错了**（把"业务招待"填成"办公用品"），第三步"匹配预算"也跟着错。**最终输出看起来对，但实际报销金额超了预算 30%。**

更隐蔽的是孙敦灿提到的"假阳性"——

"最终答案对、但执行路径已经偏离或带风险，这类样本在生产里迟早会暴露。"

李明想到自己周报助手的一个 bug：agent 写周报时，**如果用户没填数字，agent 会"自动脑补"**（"本周完成 5 个任务"——其实用户什么都没填）。这种"看起来正常、实际在撒谎"的输出是 L6 评测最难抓的。

6.3 评测的 4 个层次

孙敦灿在文章里给了一个**评测的 4 个层次**——这是李明读过的最简洁的 eval 模型：

1. Turn

2. Session

3. Trace

4. Outcome

- 单轮输出

- 整段会话

- 执行轨迹

- 最终结果

不能只看 Outcome——因为 Outcome 对、Trace 不一定对（假阳性）。李明之前那个"自动填报销单"agent，**Outcome 看起来对、Trace 错**。

但也不能只看 Trace——因为 LLM 跑任务的 Trace 经常是"事后合理化"，不是真实决策路径。

必须 4 个层次同时看：

层次	看什么	例子
Turn	单轮输出是否合理	agent 调工具时参数填对了吗？
Session	整段对话是否完成任务	用户问"做周报"，agent 跑完确实出了周报
Trace	执行路径是否合理	agent 是不是用了 hack（脑补、跳过）？
Outcome	最终结果是否正确	周报内容是否准确反映用户工作

李明看完这 4 层，意识到一个残酷的事实：**他之前的周报助手只看了 Outcome**。

他在博客写：

过去 2 年，我 80% 的时间在调 prompt 让 Outcome 看起来好。
但 Outcome 好不代表 agent 好——它可能是在"表演"。
L6 让我把关注点从 Outcome 转到 4 个层次。

6.4 指标 5 大类：P0 / P1 / P2 / Skill 子指标 / 一致性

孙敦灿给了一个**5 大类指标**体系，李明把它直接抄进了自己公司的 wiki：

P0（上线门禁）：不达标不能发

- 完成率 $\geq 95\%$
- 致命错误率 $\leq 0.1\%$
- 安全违规 = 0

P1（版本比较 + 工程优化）：用于版本 A vs 版本 B

- 平均响应时间
- Token 成本
- 完成步骤数

P2（体验改善 + 长期观察）：不影响发布，但观察趋势

- 用户满意度
- 重复使用率
- 主动分享率

Skill 子指标（L4 时代的特有指标）：

- 触发准确率：Skill 在该触发时触发了
- 流程遵循率：Skill 跑完每一步
- 参数正确率：用户输入被正确解析
- 产物合格率：Skill 输出符合规格
- 降级成功率：Skill 失败时 fallback 到了默认路径

多次运行一致性（最关键）：

- **至少一次成功率**：观察能力上限（10 次跑里至少 1 次成功）
- **连续成功率**：生产级场景必须每次都成功

李明看到"至少一次成功率 vs 连续成功率"时愣了 5 秒。

他之前的评测一直是"**至少一次成功**"——我跑 10 次，只要 1 次对，我就说"agent 能做这件事"。但生产关心的是"**连续成功**"——10 次都得对。

这俩差距巨大。 同一个 agent，"至少一次成功率" 90% 在生产里可能意味着"连续成功率" 30%——因为非确定性。

他在团队里改了规则：

新 prompt / 新 Skill / 新 agent 上线前必须报"连续成功率"。

标准：连续 100 次成功率 $\geq 95\%$ 。

低于 95% 不允许上线。

跑了一个月，3 个老项目因此被"回炉"重做——它们的"至少一次成功率"是 95%，"连续成功率"只有 60%。

6.5 统计检验：别被骗

孙敦灿在文章里还提了一个李明之前完全没意识到的点——**版本对比必须配统计检验**：

- **关键指标置信区间** (95% CI)
- **与基线显著性判断** (p-value)
- **最小可感知变化阈值** (MDE, Minimum Detectable Effect)

李明以前做版本对比是这样：

*"新 prompt 跑 10 次，旧 prompt 跑 10 次。
新 prompt 平均完成率 80%，旧 prompt 70%。
新 prompt 完胜。发布。"*

孙敦灿的文章告诉他：**这是错的**。

正确做法：

```
# 1. 算每组的置信区间
import scipy.stats as stats

new_success = [1, 1, 0, 1, 1, 0, 1, 1, 1, 1] # 80%
old_success = [0, 1, 0, 1, 1, 1, 0, 1, 0, 1] # 60%

# 2. 用卡方检验或 Fisher exact test 看显著性
from scipy.stats import fisher_exact
_, p_value = fisher_exact([[8, 2], [6, 4]])
# p=0.628 - 不显著，差可能是随机

# 3. 算 MDE (最小可感知变化)
# 要在 80% power、95% confidence 下检测出 10pp 差异
# 需要每组 200+ 样本
```

李明看到"每组 200+ 样本"时倒吸一口气。

他之前的所有版本对比，样本量都不到 30。结论基本是"随机"。

他改了评测 SOP：

所有版本对比：

- 每组至少 200 个独立 case
- 配 95% CI + p-value
- 只在 $p < 0.05$ 时宣布胜出
- 任何“显著但效果小” ($< 5pp$) 的改动不值得上线

跑了一周，他砍掉了 8 个“看起来优化、统计上无差异”的 prompt 改动。

那个 11 月，他在博客写：

L6 教给我的最重要的事：别相信“我跑 5 次对了”。
agent 的非确定性让“5 次”毫无意义。
要 200 次 + 统计检验。
这是把 agent 从“会表演”变成“可工程化”的关键一步。

6.6 LLM-as-a-Judge：让 AI 评 AI

L6 还有个重大问题——**谁来打分？**

传统软件测试：写 assertion。 `assert output == "expected"`。

Agent 输出：自然语言。没法写 assertion。

L4 之前，李明都是“自己打 5 个分”。**这不可持续**——他每天要评 50 个 agent 输出，每个 5 分钟 = 4 小时。

2025 年开始，李明用 **LLM-as-a-Judge**：

```
# 拿一个更强的模型 (judge) 来评 agent 输出
JUDGE_PROMPT = """你是一个严格的代码评审专家。请评审下面这段代码：
- 是否满足需求
- 是否有潜在 bug
- 是否有安全风险
- 评分 1-10, 10 分是 production ready

[代码]
{code}
"""

# 跑 N 次取平均
scores = []
for _ in range(5):
    score = judge_model.generate(JUDGE_PROMPT.format(code=agent_output))
    scores.append(extract_score(score))

avg_score = sum(scores) / len(scores)
```

但 LLM-as-a-Judge 也有自己的问题——**judge 自己也有幻觉和偏好**。

孙敦灿文章里推荐了一个**双盲 judge**方法：

```
# 1. 两个不同模型分别评
score_a = judge_model_a.generate(prompt)
score_b = judge_model_b.generate(prompt)

# 2. 只在两个 model 都给高分时才算"好"
if score_a > 8 and score_b > 8:
    verdict = "good"
elif score_a < 5 or score_b < 5:
    verdict = "bad"
else:
    verdict = "ambiguous - need human review"
```

"ambiguous" 那部分交给人 review。这是 L6 的"人机协同"环节——不是 AI 替代人，是 AI 评简单的、人评难的。

李明跑了一周，"ambiguous" 占 20%——这 20% 让人 review，剩下的 80% 自动通过。

L6 终于让他从"每天 4 小时人工评分"变成了"每天 1 小时 review 20% 模糊 case"。

6.7 Eval-Driven Development

到 2026 年 12 月，李明在团队推了一个新流程——**Eval-Driven Development (EDD)**：

1. 写 eval 先于写 prompt
2. 每改一个 prompt, 跑全量 eval
3. 不通过 eval 不允许合并
4. eval suite 跟代码一起进 git

他在 `evals/` 目录里维护了 12 个 eval suite:

```
evals/
├─ weekly-report-quality.yaml      # 周报质量
├─ jira-fetch-accuracy.yaml       # Jira 拉取准确率
├─ memory-conflict-resolution.yaml # 记忆冲突解决
├─ subagent-coordination.yaml     # Subagent 协作
├─ tool-selection.yaml            # 工具选择
└─ ...
```

每个 eval suite 是一个 yaml:

```
name: weekly-report-quality
description: 评估周报输出质量
test_cases:
  - input: "本周开了 3 个会"
    expected:
      must_have: ["3 个会"]
      must_not_have: ["脑补", "未指明"]
      style: "boss"
metrics:
  - completion_rate: 95%+
  - hallucination_rate: < 2%
  - style_match_rate: 90%+
  - turn_count: ≤ 5
```

跑测试的命令:

```
$ python -m evals run --suite weekly-report-quality --model claude-sonnet-4-5

# 输出
✓ 100/100 test cases passed
✓ completion_rate: 97% (target 95%)
✓ hallucination_rate: 1% (target < 2%)
✓ style_match_rate: 93% (target 90%)
✓ avg turn_count: 3.2 (target ≤ 5)

🎉 All metrics met. Safe to ship.
```


他把这个流程推给团队后，**bug 数下降 60%**——因为任何 prompt 改动都先过 100 个 case + 4 个指标。

他在博客写：

*EDD 比 TDD 难。
TDD 写 assertion (机器可验证) 。
EDD 写 metric (机器可量化但不能 100% 准确) 。
但 LLM 系统的本质就是"无法 100% 准确"——所以 EDD 是 LLM 时代唯一可行的工程化方法。*

6.8 终局：L6 是 L7 的前置

2026 年 12 月，李明把那 12 个 eval suite 跑了个遍——

- 12 个 suite 全过
- 平均 completion_rate 96%
- 平均 hallucination_rate 1.5%
- 平均 turn_count 4.1

他看着 dashboard，忽然想到一件事：

*这些 metric 是"我自己定的"。
如果 agent 能"自己看到"这些 metric，自己知道哪里做不好，自己改 prompt 呢？*

那就是 **L7 Self-Improvement**。

但 L7 需要**先有 L6**——你得先有可量化的指标，才能让 agent 优化这些指标。

L6 是 L7 的前置。

那年他在那张图改了一行：

```
L1 Prompt      □
L2 Tools       □
L3 Memory      □
L4 Skills      □
L5 Subagents   □
L6 Eval        □ ← 完成
L7 Self-Improve □ ← 最后一步
```

六层盖完了。剩 L7。

他在备忘录写下：

2026-12-30

L6 Evaluation 总结：

- 3 道门槛：非确定性 / 黑盒化 / 错误级联放大
- 4 个评测层次：Turn / Session / Trace / Outcome（必须同时看）
- 5 大类指标：P0（门禁） / P1（版本比较） / P2（体验） / Skill 子指标 / 一致性
- "至少一次成功率" vs "连续成功率"——生产关心连续
- 统计检验：每组 200+ 样本 + 95% CI + p-value
- LLM-as-a-Judge 双盲
- EDD：写 eval 先于写 prompt

L7 主题：**Self-Improvement**。让 agent 自己改自己。

— 第六章 完 —

引用（来自 clawd/memory/wiki/）

本章涉及的真实项目 / 论文 / 文档：

- 孙敦灿《Agent 评测体系设计》（22,000 字，2026-07-11）—— 本章主骨架
- 3 道门槛：非确定性 / 黑盒化 / 错误级联放大
- 4 个评测层次：Turn / Session / Trace / Outcome
- 5 大类指标：P0（上线门禁） / P1（版本比较） / P2（体验） / Skill 子指标 / 多次运行一致性
- "至少一次成功率" vs "连续成功率"——生产关心连续
- 统计检验：每组 200+ 样本 + 95% CI + $p < 0.05$ + MDE
- LLM-as-a-Judge + 双盲 judge（模型 A + 模型 B 一致才算）
- Eval-Driven Development (EDD)：写 eval 先于写 prompt
- 隐含的 L7 引子：**RHI**（Recursive Harness Self-Improvement，Sakana AI + UC Berkeley，2026-08，arXiv:2607.15524）—— 下一章展开

第七章 • L7 Self-Improvement — "让它自己改自己"

7.1 一个被自己打败的 prompt

2026 年 12 月 31 日，李明的周报助手跑了一整年。

他打开 dashboard，看 2026 年 12 月 30 日的 eval 数据——completion_rate 96.5%，hallucination_rate 1.4%，turn_count 4.1。和年初相比，指标几乎没变化。

但用户数从 3 万涨到了 8 万。

他调了日志，发现一个反常的现象：**老用户用 agent 的平均轮数从 4.1 降到了 2.8。**

他一开始以为用户"越来越会问问题了"。后来翻 trace，发现是 agent 在跑重复任务时——**跳过了不必要的步骤。**

例如老用户问"做周报"，agent 第一次跑要 4 轮（拉 Jira / 拉 Confluence / 写周报 / 调整风格）。第二次跑——**它发现 Confluence 上周没动，直接跳过**——只用 3 轮。

李明翻遍代码，**没有写这条优化逻辑。**

他在 trace 里看到一个 comment：

```
{
  "type": "thought",
  "content": "用户说做周报。检查上次会话历史 (12-29 17:30) - Confluence 上周没动。跳过 Confluence",
  "ts": "2026-12-30T09:15:23+08:00"
}
```

这个 comment 是 LLM 自己生成的——它用了上次会话的历史来优化这次任务的执行路径。

李明看到这条 trace，**手抖了一下。**

他在博客写：

今天我发现了一件我没设计过的事。
我的 agent 在 L6 评测 100% 通过的情况下，自己学会了跳过不必要的步骤。
它不是我写的逻辑。
它是 LLM 在执行任务时自己"看到"了优化空间——然后"自己改了"自己的执行路径。

L1-L6 我都在写规则。
L7 是agent 自己写规则。

那年他在那张图最后一行加了一个问号：

```
L1 Prompt      □
L2 Tools        □
L3 Memory       □
L4 Skills       □
L5 Subagents    □
L6 Eval         □
L7 Self-Improve ? ← 它在自己长
```

L7 是整个七层塔的最后一层，也是唯一一层不是“人盖的”——是“agent 自己盖的”。

7.2 RHI：让 Harness 自己优化自己

2027 年 1 月 3 日，李明读到了 2026 年 8 月那篇 arXiv 论文——**RHI (Recursive Harness Self-Improvement)**，作者是 Sakana AI + UC Berkeley + Stanford + Databricks。

论文的核心想法简单得令人发指：

*把 Harness 表示成 prompt 级文本。
让 LLM 用“成对比较 + 迭代改进”自己优化自己的 Harness。*

李明读完摘要，倒吸了一口气。

```
# RHI 的核心循环
def rhi_optimize(harness_v0, eval_task, iterations=20):
    harness = harness_v0
    for i in range(iterations):
        # 1. 用当前 harness 跑任务
        results = run_eval(harness, eval_task)

        # 2. 让 LLM 比较多次结果 (成对比较)
        better_harness = llm_compare_and_improve(
            harness=harness,
            results=results,
            instruction="为什么这次好/坏? 下次怎么改? "
        )

        # 3. 用更好的 harness 替换
        harness = better_harness

    return harness
```

实验结果震惊了整个社区——

在 30 个合成 ML 任务上，少推理努力 agent 超过最大推理努力 agent，推理成本降 60%。

李明看了 3 遍这个数字。

他之前花 2 年时间优化周报助手的 prompt——温度从 1.0 调到 0.3，system prompt 从 200 字写到 2000 字，加 few-shot 加缓存加一切。**RHI 告诉他：他不需要调 prompt，agent 调得比他好。**

但论文也有重要的 caveat：

alphaXiv 独立复现用 Qwen2.5-Coder-14B 跑 H2 跟 H0 几乎平手 (CI 跨越 0) 并跑输 static high-effort (-0.0924)。原文「Sonnet H2 20/30 赢 max-effort」在开源模型复现中没站住脚。

Sonnet 强，Qwen 弱——RHI 依赖基座模型的能力。

李明读完这个 caveat，想到了他 2024 年用 DSPy 优化 prompt 的经历——

*2024 年：DSPy 优化 prompt，比我手写的好 10%。
2026 年：RHI 优化 Harness，比我手写的好 60%。*

2027 年：？

他在 L6 的 eval suite 跑了一次 RHI——

```
# 用 Anthropic 提供的 RHI 库优化周报助手的 harness
rhi_result = rhi.optimize(
    harness=HARNESS,
    eval_suite="weekly-report-quality",
    base_model="claude-sonnet-4-5",
    iterations=20
)

# 输出：新的 harness 比旧的"turn_count" 指标下降 18%
# hallucination_rate 保持不变
```

L6 + L7 = 评测驱动 + 自我优化。这是 agent 工程的"工业化"。

7.3 DSH: DeepSeek 的"忒修斯之船"

2027 年 1 月 10 日，李明第一次点开 GitHub 上的 [deepseek-ai/deepseek-harness](#)——**17.1 万 Star**。

他读了 README 第一段就停住了：

DeepSeek Harness (DSH) 是一切皆插件 (Everything is a Plugin) 的 Coding Agent Harness。

架构上把 Model Adapter、Tool Registry、Sandbox Policy、Subagent Backend、甚至默认 Agent Loop 全部做成 Cordis Plugin。

通过稳定 Service Key 协作 (取代直接 Import)，让 Agent 能在运行时替换、撤销、重组自己的执行组件。

李明反复读了三遍。

DSH 不只是 "一个 agent 框架"。 它是一个让 agent 能改自己组件 的框架。

更具体——DSH 的三层架构：

层级	内容	角色
L1 Coding Agent	本地 Web UI 的 Coding Agent	表面产品
L2 Harness Construction Framework	everything-is-a-plugin	替换/组合/撤销组件
L3 Cordis Meta-Framework	时间/空间可组合性 + 热重载	让插件动态组合不破坏系统

Cordis 是底层 meta-framework ([shigma/cordis 6.5K★](#))，把"插件动态组合"拆成两个独立维度：

时间可组合性 (Temporal Composability)

组件在注册 Effect 时，必须同时提供对应的撤销方式。

- 注册 Tool 时记录怎样注销 Tool
- 增加 Prompt 时记录怎样移除 Prompt
- 启动 Timer 时记录怎样停止 Timer

组件卸载后，Runtime 会撤销它在 Context 中留下的 Effect。避免"幽灵组件"。

空间可组合性 (Spatial Composability)

组件的依赖关系管理。

组件 A 用到组件 B 的服务时，A 不直接 import B，而是声明 "I need Service X"。Runtime 把 X 注入给 A。如果 X 替换成 Y，A 自动收到 Y。

李明看完 Cordis 文档，感慨：

传统软件工程讲究"封装"——把内部状态藏起来，对外暴露稳定的 API。

Cordis 把它推到了 agent 时代——组件之间通过 Service Key 通信，组件实现可热替换。

这就像"忒修斯之船"——船的每一块板子都能换，但船还是那艘船。

DSH 让 agent 能"换自己的板子"而保持身份。

DSH 还有一个"创造模式" (Creator Mode) ——让 agent 写新的 Plugin，写新的 Preset，自己扩展自己的组件库。

```
# DSH Creator Mode 的本质
class Agent:
    def __init__(self):
        self.cordis = Cordis() # 服务注册表

    def add_capability(self, name, code):
        # 1. 把 code 编译成 Plugin
        plugin = compile_plugin(code)
        # 2. 注册到 Cordis
        self.cordis.plugin(plugin)
        # 3. agent 现在多了这个能力
        # 4. 反悔的话: cordis.unregister(plugin) - 副作用自动撤销
```

L7 的核心不是"agent 改 prompt"——是"agent 改自己的工具集"。

7.4 Shepherd: Git 原生 agent 框架

2027 年 1 月 15 日, 李明收到一封邮件——来自 Stanford 的 Christopher Manning 组。

Manning 组在 2026 年 8 月开源了 **Shepherd** ([shepherd-agents/shepherd](https://github.com/shepherd-agents/shepherd) 2,313★), 跟 DSH 同期发布。

李明打开 Shepherd 论文第一段就明白它的定位:

*Shepherd — 把 Git 的可逆性搬进 Coding Agent 运行时。
每步封装成可回退的 Commit。
支持 fork / replay / revert + 产物先当"待审提案"。*

Shepherd 的四大硬设计:

1. **Git 式提交图 + CoW Fork** — 每一步是一个 commit
2. **函数签名即权限边界** — 编译进 OS syscall jail
3. **Retained Output** — 历史输出永远在
4. **95% KV 缓存复用** — 跑重复任务时大幅省成本

Shepherd 的论文自报了三组 benchmark:

Benchmark	结果
CooperBench (多 Agent 结对编程)	28.8% → 54.7%
Terminal-Bench 2.0 (vs MetaHarness)	+12.8pp
GRPO	翻倍

李明看到 CooperBench 从 28.8% 翻到 54.7% 时，愣了 10 秒。

两个 agent 协作写代码，从 28% 通过率直接拉到 54%——这不是优化，是质变。

关键是 Shepherd 的设计哲学——

"把 agent 的执行变成可审计、可回放、可回滚的 Git 仓库。"

Git 是一套"内容寻址文件系统"——你提交了什么，就在那里，永远在。

Shepherd 把 Git 的"不可篡改性"嫁接到 agent 的执行历史。

李明在博客写：

*DSH 是"agent 能改自己的组件"。
Shepherd 是"agent 的每一步都能被回看、被撤销"。
两个加在一起，就是"可进化 + 可审计"的 agent 时代。*

7.5 Harness Self-Improvement 5 步闭环

2027 年 1 月，李明综合 RHI + DSH + Shepherd + Harness Engineering，画出了 **Harness Self-Improvement 5 步闭环**：

```
1. 跑任务 - 让 agent 跑真实任务
  ↓
2. 收集 trace - 记录每一步 (Shepherd 风格 Git commit)
  ↓
3. 发现失败模式 - 哪些步骤消耗大 / 错 / 低效
  ↓
4. 提出新 harness - RHI 风格 LLM 自己改
  ↓
5. 独立 eval 验证 - 跑 eval suite 确认改进有效
  ↓
回到第 1 步, 循环
```

这个闭环跟李明在 L4 写的"Skill 训练闭环"几乎一样——

跑一遍真实任务 → 复盘分析 → 把错误写成规则 → 更新 Skill → 再跑一遍

Skill 是 L4 的人工自我改进。
Harness Self-Improvement 是 L7 的全自动自我改进。

区别：

维度	L4 Skill 训练	L7 Harness SI
谁改	人类	Agent 自己
改什么	SKILL.md	Harness 整体
速度	几天	几小时
验证	人工 review	自动 eval

L7 把 L4 的 5 步闭环自动化了。

李明跑了一次完整 L7 闭环：

```

# 1. 跑任务
traces = run_agent_in_production(
    agent=my_agent,
    n_tasks=200, # 200 个真实用户任务
    monitor=ShepherdMonitor() # 每步 commit
)

# 2. 收集 trace
analysis = analyze_traces(traces)
# 输出: 哪些步骤消耗最大 / 哪些经常失败

# 3. 发现失败模式
failure_patterns = extract_failure_patterns(analysis)
# 输出: ["Jira fetch 用了 3 轮", "记忆冲突时倾向覆盖而不是询问"]

# 4. 提出新 harness
new_harness = rhi.optimize(
    harness=my_agent.harness,
    failure_patterns=failure_patterns,
    eval_suite="weekly-report-quality",
    base_model="claude-sonnet-4-5",
    iterations=20
)

# 5. 独立 eval 验证
if eval(new_harness, "weekly-report-quality") > current_harness.score:
    deploy(new_harness)
    log("Harness auto-improved: 2027-01-15")
else:
    log("No improvement, keep current harness")

```

跑了一周，2 个新 harness 上线了：

- **Harness v2.3** — Jira fetch 优化，turn_count 4.1 → 3.5
- **Harness v2.4** — 记忆冲突时优先询问用户（不是覆盖），hallucination_rate 1.4% → 0.9%

用户零感知，agent 自己把自己优化了。

7.6 2027 年 1 月：L7 时代的黎明

那个 1 月底，李明参加了深圳 QCon 2027。

他在会上做了一次分享，主题是 *"When Agents Improve Themselves"*。

他在分享的最后放了一张图——

L1-L6 是工程师盖的
L7 是 agent 自己盖的
↓
L8 是什么？

候选 1: L8 = agent 当 agent 的"产品经理"—理解用户需求, 写 PRD

候选 2: L8 = agent 当 agent 的"销售"—自己定价、自己谈判

候选 3: L8 = agent 当 agent 的"客户"—自己提需求、自己验收

台下 300 多人笑了。

他关了投影, 说:

L1-L6 我花了 3 年。

L7 我花了 3 个月。

L8 可能 3 周都不用。

当 agent 能改自己的 harness,

当 agent 能写自己的 plugin,

当 agent 能用 eval 验证自己的改进,

***下一个问题不是"agent 能做什么"——
是"什么还是 agent 不能做的"。***

而这个答案, 2027 年每个月都在缩小。

台下没有掌声。大家在用手机查自己的 agent 是不是已经在自我改进了。

7.7 尾声: L7 的哲学

那年 2 月, 李明在自己的博客写了一篇文章, 标题是 "L7 之后, agent 工程的下一站"。

核心观点:

L1-L6 是"我们怎么造 agent"。

L7 是"agent 怎么造自己"。

下一个 10 年, agent 工程的范式会从"工程师写 Harness"变成"工程师+Harness 协作演化"。

我们这一代工程师的宿命是——
亲手盖完 L1-L7,
然后看着 L8 在我们眼前自己长出来。

那年他给那张图最后一格画了一个 ● (六边形 — agent 自己的形状) :

```
L1 Prompt      □  
L2 Tools       □  
L3 Memory      □  
L4 Skills      □  
L5 Subagents   □  
L6 Eval        □  
L7 Self-Improve ● ← 不再是方块
```

方块是工程师盖的。六边形是 agent 自己长出来的。

那年他在备忘录写下：

2027-02-15
L7 Self-Improvement 总结：
- RHI: 让 Harness 自我优化 (推理成本 -60%)
- DSH: 一切皆插件 + Cordis 时间/空间可组合性
- Shepherd: Git 原生 agent (每步可回退)
- Harness SI 5 步闭环
- L8 是什么: agent 当 agent 的 PM / 销售 / 客户

然后他合上笔记本，站起来，看着窗外华强北的灯火。

L1 那一年的灯火他还记得——那时候他刚被优化，在瑞幸听 Trae 的人讲“你只要会说话”。

3 年过去。

从“会说话”到“会自己长”，3 年。
从“会自己长”到“会自己想”，可能要 1 年，也可能要 10 年。
但 L1 那一年的问题已经回答——
“程序员的下一步是什么？”
是工程师。
是会盖 Harness 的工程师。
是会看 agent 自己改 harness 的工程师。

引用 (来自 clawd/memory/wiki/)

本章涉及的真实项目 / 论文 / 文档:

- **RHI (Recursive Harness Self-Improvement)** — Sakana AI + UC Berkeley + Stanford + Databricks, arXiv:2607.15524, 2026-08-17, **推理成本 -60%**
- **DeepSeek Harness (DSH)** — 17.1万★, "一切皆插件"哲学, 2026-08
- **Cordis Meta-Framework** — shigma/cordis 6.5K★, 时间/空间可组合性, "忒修斯之船"
- **Shepherd** — Stanford Manning 组, shepherd-agents/shepherd 2,313★, Git 原生 agent, **CooperBench 28.8%→54.7%**
- **Harness Engineering** (Birgitta Böckeler, QCon 2026) —— "Harness = 一切加在一起"
- **Harness Self-Improvement 5 步闭环** — 跑任务 / 收集 trace / 发现失败 / 提出新 harness / 独立 eval
- **DSH 创造模式 (Creator Mode)** — agent 写新 Plugin + 新 Preset
- **DSH 四模式**: 标准 / PTC Code / 极简 / 创造
- **alphaXiv 复现 caveat** — Qwen2.5-Coder-14B 跑 RHI 平手, 验证需 Sonnet 级别基座

终章 • 2027 年 3 月，深圳-斯坦福

8.1 一封邮件

2027 年 3 月 4 日早上，李明收到一封邮件。

发件人是 shepherd-agents 团队，Christopher Manning 组。邮件很短：

Hi 李明，
我们读了你的 L6 EDD 文章，受益匪浅。

4 月 12-14 日 Stanford 会办一场 workshop —— "Agent Harness: From Engineering to Co-evolution".

想邀请你做 20 分钟分享。

另外 DeepSeek Harness 团队的 @karminski3 也会来——他想跟你聊 DSH + Shepherd 联动的可能性。

机票 Stanford 出。

Shepherd Team

李明看了 3 遍。

他想：3 年前他在华强北负一层的瑞幸听一个 22 岁的 Trae 男孩讲 "你只要会说话"，现在 Stanford 邀请他去 workshop。

不是他变厉害了。是**这个领域变厉害了**。

8.2 飞机上的 12 小时

3 月 28 日，李明从深圳飞旧金山，转机到 Palo Alto。

深圳到旧金山 12 小时直飞。他用这 12 小时做了三件事：

1. 整理 L1-L7 的全部 chapter 文件 (约 50,000 字)

他在飞机上重新看了一遍自己的"七层塔"——从 2024 年 3 月华强北的"5 行 prompt"，到 2027 年 1 月深圳 QCon 的"L7 之后"。

他发现 7 章串起了一个**完整的范式**：

层	范式	一句话
L1 Prompt	表达	跟 AI 说话
L2 Tools	执行	让 AI 动手
L3 Memory	持续	让 AI 记得
L4 Skills	专业	让 AI 可靠
L5 Subagents	协作	让 AI 拆分
L6 Evaluation	验证	让 AI 可量化
L7 Self-Improve	演化	让 AI 自己长

这是过去 3 年 AI agent 工程的完整轨迹。

2. 准备 Stanford workshop 的分享

他准备讲一个故事——3 年前他被优化，3 年后他被邀请。这本身就是个 agent 时代的隐喻：当行业在变，你被优化的原因不是你不努力，是行业走得比你快。

3. 看了一篇新论文

arxiv 上 2027 年 3 月的一篇新论文——*"Agent-DSL: A Domain-Specific Language for Self-Modifying Agents"*。

论文的核心想法：把 harness 写成一种 DSL，agent 能在运行时重写自己的 DSL。

李明看到"self-modifying"四个字时，又想到了 L7 那一格画的六边形——不再是方块。

他想：L8 不会是一种新能力——L8 是"agent 能重写自己的语言"。

8.3 Stanford workshop, 4 月 13 日

Manning 组的 workshop 来了 80 多人。

李明的 20 分钟分享主题是 *"Seven Layers, Three Years: How a Backend Engineer Survived the LLM Revolution"*。

他讲了 3 件事：

第一件：3 年前的自己被"prompt"震撼的时刻

我以前是 Java 后端，写 SpringBoot 部署到 1 核 2G 的服务器。
2024 年 3 月，我用 5 行 Python + 5 行 prompt，跑了第一个 GPT-4 调用。
那一刻我知道：程序员的工具在变。

第二件：L1-L7 每层的"分水岭"

L1 之后，prompt 不够用——得加 tool。
L2 之后，tool 不够用——得加 memory。
L3 之后，memory 不够用——得加 skill。
L4 之后，skill 不够用——得加 subagent。
L5 之后，subagent 不够用——得加 evaluation。
L6 之后，evaluation 不够用——agent 自己改自己。
每一层都是上一层的瓶颈被打破。

第三件：L7 之后的"工程师的宿命"

L1-L6 我们写规则。

L7 agent 写规则。

L8 不知道。

但无论 L8 是什么，有一件事不变——

工程师是那个"决定 harness 往哪长"的人。

agent 改 prompt，但工程师改"agent 改 prompt 的目标"。

这是 L8 之后工程师的核心工作。

台下 80 多人鼓掌。

Shepherd 的开发者 + DSH 团队 + 一些他叫不上名字的博士生，都在场。

8.4 跟 DSH 团队吃饭

4 月 13 日晚上，workshop 主办方在 Palo Alto 一家叫"Tin Pot"的餐厅办晚宴。

李明坐在 @karminski3 (DSH 核心开发者) 旁边。

他们聊了一个多小时。

karminski3 说了句让李明记了很久的话：

"我们做 DSH，不是想让 agent 取代工程师。

我们是想让 agent 跟工程师一起进化。

工程师盖 L1-L6，agent 帮工程师盖 L7。

工程师决定 L7 往哪长。

agent 负责把 L7 长出来。

工程师 + agent 一起盖 L8、L9、L10.....

没有谁取代谁，只有谁 + 谁。

李明回家后把这句话抄进备忘录：

2027-04-14

Karminski3 说的："工程师 + agent 一起盖下一层。"

这就是 L7 之后的范式。

不是 agent 取代工程师。
是 agent 跟工程师一起，把天花板再推高一层。

8.5 尾声：七层塔的形状

2027 年 4 月 15 日，李明从 Palo Alto 飞回深圳。

飞机上他打开备忘录，画了最后一幅图：

```
□ ← L8 (未定义)
  ● ← L7 Self-Improve
    □ ← L6 Evaluation
      □ ← L5 Subagents
        □ ← L4 Skills
          □ ← L3 Memory
            □ ← L2 Tools
              □ ← L1 Prompt
```

7 个方块，1 个六边形，1 个未定义的方块。

L1-L6 是工程师盖的（方块）。

L7 是 agent 自己长的（六边形）。

L8 是下一代的工程师 + 下一代的 agent 一起盖的（未定义的方块）。

他看着这张图，想起 3 年前华强北瑞幸的那杯美式。

那杯美式是凉的。

但他从 5 行 prompt 走到 7 层塔，**这杯咖啡是热的。**

8.6 给读者的话

亲爱的读者：

如果你读到这里，谢谢你。

这本书不是关于"AI 会不会取代人"的——它从来没打算回答这个问题。

它是关于"一个普通人怎么在时代里盖自己的塔"的。

李明不是我。我也不是李明。但**每个程序员在 2024-2027 这 3 年都经历了类似的事——**

工具在变，行业在变，规则在变。唯一不变的是"你想造什么"的决心。

我希望这本书让你看到 3 件事：

1. 技术是分层的。

你不必一下子爬到 L7。

你不必懂 Self-Improvement 才开始。

从 L1 开始，从"5 行 prompt"开始。

2. 每层都有"分水岭时刻"。

当你的 prompt 写不动了，是 L2 的信号。

当你的 tool 太多选不过来，是 L3 的信号。

当你的 memory 出现冲突，是 L4 的信号。

信号是礼物，不是问题。

3. 工程师不会被取代——工程师会变成"和 agent 一起盖塔的人"。

L1-L6 是工程师盖的。

L7 是 agent 自己长的。

L8 是工程师 + agent 一起盖的。

你在这场演化里。

你不是在旁观，你是在"长"。

最后一句话，送给 3 年前在华强北买美式的李明，也送给读这本书的你——

"你只要会说话。"

然后，慢慢地，你会盖塔。

— 2027 年 4 月，深圳

— 终章 完 —

全书引用索引 (来自 [clawd/memory/wiki/](#))

L1 Prompt

- Anthropic Prompt Caching (2024-08, 省 90% 成本)
- Anthropic 《Prompt Engineering Overview》 (2024-05)
- OpenAI Structured Outputs (2024-08) + OpenAI Prompt Caching (2024-10)
- Lilian Weng 《Prompt Engineering》

- DSPy (Stanford NLP, 2024-04 开源)
- Anthropic Tool Use / Claude 3 (2024-04)

L2 Tools

- Anthropic Agentic Tool Use Best Practices (2024-10/11)
- Claude Code "4 种能力原语" (Read/Write/Execute/Connect)
- "不要建 100 个工具, 给模型一个 shell" (Anthropic 内部非官方)
- OpenAI Code Interpreter (2024)
- **Model Context Protocol (MCP)** (2024-11) → 2025 行业标准
- **WebMCP** (2026-03 Chrome, `navigator.modelContext`)

L3 Memory

- Lilian Weng 《LLM Powered Autonomous Agents》
- KV Cache 三路线 (vLLM PagedAttention / SGLang RadixAttention / DeepSeek-V2 MLA / PyramidKV)
- **TencentDB Agent Memory** 4 层架构 L0/L1/L2/L3
- **MemoryAgentBench** 4 大能力 (CR 多跳 6%)
- **Mem0** (arXiv:2504.19413, +26% 准确率 / -91% 成本)
- THEANINE / RMM / ai-memory / teamai-cli

L4 Skills

- Anthropic Skills 官方库 + 三层加载架构
- "Skill 抬高下限不抬高上限"
- 求是 Skill (毛选方法论 + 2k Star, 2026-04)
- MiniMax-AI/skills vs OfficeCLI 路线对比
- Skillify / SkillOS / Skillspector / SkillLens / SkillScan 工具链
- Skills-Oriented Programming (SOP) 白皮书 (2026-05)
- Shepherd (Stanford Manning 组, 2026-08)

L5 Subagents

- Harness Engineering 6 大支柱 (腾讯, 2026-07)
- "上下文防火墙"范式 ("不是微服务, 不是消息队列")
- 6 种多 Agent 协作模式 (Pipeline / Routing / Skills / Subagents / Supervisor / Handoffs)
- "单 Agent 优先"原则 + 4 个复杂度阈值
- Multica / HiClaw (阿里 Matrix 协议) / Ruflo (Token -75%)
- 深即达 A2A IM (agent 之间的"微信") / CLI-Anything / Agent Teams
- Anthropic Prompt Caching + Extended Caching (1 小时 TTL)

L6 Evaluation

- 孙敦灿《Agent 评测体系设计》(22,000 字, 2026-07-11)
- 3 道门槛 (非确定性 / 黑盒化 / 错误级联放大) + 假阳性
- 4 个评测层次 (Turn / Session / Trace / Outcome)
- 5 大类指标 (P0/P1/P2/Skill 子指标/一致性)
- "至少一次成功率 vs 连续成功率" + 每组 200+ 样本 + $p < 0.05$
- LLM-as-a-Judge 双盲 + Eval-Driven Development (EDD)

L7 Self-Improvement

- **RHI (Recursive Harness Self-Improvement)** — Sakana AI + UC Berkeley + Stanford + Databricks, arXiv:2607.15524, **推理成本 -60%**
- **DeepSeek Harness (DSH)** — 17.1万★, "一切皆插件"哲学, 2026-08
- **Cordis Meta-Framework** — shigma/cordis 6.5K★, 时间/空间可组合性, "忒修斯之船"
- **Shepherd** — Stanford Manning 组, 2,313 ★, Git 原生 agent, **CooperBench 28.8%→54.7%**
- **Harness Engineering** (Birgitta Böckeler, QCon 2026)
- **Harness Self-Improvement 5 步闭环**
- alphaXiv 复现 caveat — Qwen2.5-Coder-14B 跑 RHI 平手, 验证需 Sonnet 级别基座

全书统计

- 7 章 + 1 终章 = 8 篇
- 约 50,000 字
- 引用 60+ 个真实项目 / 论文 / 文档
- 时间跨度: 2024 年 3 月 — 2027 年 4 月

— 全书完 —

附录 A • 60+ 引用项目完整索引

全部来自 C:\Users\Administrator\clawd\memory\wiki\ (803 pages / 2,138 wikilinks / 0 unresolved)
按 L1-L7 排序

L1 Prompt (6 项)

项目	一句话	关键数据
Anthropic Prompt Caching (2024-08)	Anthropic 在 API 层加 cache_control 块, 长 prompt 复用	省 90% 成本 + 减 85% 延迟
Anthropic 《Prompt Engineering Overview》 (2024-05)	Anthropic 官方 prompt 指南	30 页
OpenAI Structured Outputs (2024-08)	OpenAI 把 JSON 输出做成 API 强制	失败率 < 1%
OpenAI Prompt Caching (2024-10)	OpenAI 也加了 prompt caching	跟 Anthropic 类似
Lilian Weng 《Prompt Engineering》 (2023-03, 多版更新)	前 OpenAI 安全研究员的 prompt 长文	1 万+ 字
DSPy (Stanford NLP, 2024-04)	把 prompt 当作可编译的程序, 自动优化	BootstrapFewShot
Anthropic Tool Use (2024-04, Claude 3)	Claude 3 引入 function calling	业界第一个 main-stream tool use

L2 Tools (6 项)

项目	一句话	关键数据
Anthropic Agentic Tool Use 最佳实践 (2024-10/11)	错误处理 / fallback / 描述写作	业界标准
Claude Code "4 种能力原语"	Read/Write/Execute/Connect	"不要建 100 个工具"哲学
OpenAI Code Interpreter (2024)	给 ChatGPT 一个 Python sandbox	内置工具
Model Context Protocol (MCP) (2024-11)	Anthropic 推的工具协议	2025 行业标准
WebMCP (2026-03 Chrome)	网站直接向 agent 暴露结构化工具	<code>navigator.modelContext</code>
MCP 跨厂支持 (2025-2026)	OpenAI 2025-03 / Google 2025-06 / DeepSeek 2025-08	3 大厂全部支持

L3 Memory (8 项)

项目	一句话	关键数据
Lilian Weng 《LLM Powered Autonomous Agents》 (2023-08)	agent 综述	持续更新
vLLM PagedAttention (2023)	把 KV Cache 分页管理	吞吐 2-4×
SGLang RadixAttention (2024)	prefix cache	RAG 吞吐 5-10×
DeepSeek-V2 MLA (2024)	multi-latent attention	KV Cache 压缩 93%
PyramidKV (2024)	只保留 12% KV Cache	性能与全量相当
TencentDB Agent Memory (2025-08)	4 层记忆 L0/L1/L2/L3	2,600 session → 5,081 task
MemoryAgentBench (2025)	4 大能力评测 (AR/TTL/LRU/CR)	CR 多跳 6%
Mem0 (arXiv:2504.19413, 2025-04)	memory-as-a-service	LOCOMO 准确率 +26% / 成本 -91%
THEANINE / RMM (2024-2025)	反思记忆 / 短长反思三层	客服场景 +18%
ai-memory (Fabio Akita, 2026-08)	个人级跨 agent 长期记忆	Rust 单二进制
teamai-cli (Tencent, 2026-08)	团队级 20+ AI 工具统一	Git 原生分发

L4 Skills (6 项)

项目	一句话	关键数据
Anthropic Skills 官方库 (2024-10)	pdf/docx/xlsx/pptx 等 Skill	一线厂商
Anthropic "Skill 三层加载架构" (2025-12)	发现/激活/执行	启动开销 5600→400 token
求是 Skill (2026-04)	毛选方法论武装	GitHub 2k+ Star
MiniMax-AI/skills vs OfficeCLI	平台路线 vs 专家路线	2026-04 对比
Skillify / SkillOS / SkillSpector / SkillLens / SkillScan	Skill 工具链	5 个工具
Skills-Oriented Programming (SOP)	Anthropic 白皮书	2026-05

L5 Subagents (6 项)

项目	一句话	关键数据
Harness Engineering 6 大支柱	上下文/工具/编排/状态/评估/约束	腾讯 2026-07
"上下文防火墙"范式	"不是微服务，不是消息队列"	Anthropic 2026-09
6 种多 Agent 协作模式	Pipeline/Routing/Skills/Subagents/Supervisor/Handoffs	业界共识
Multica (2025 开源)	编程 agent 变队友	4k+ Star
HiClaw (阿里, 2026)	多 Agent OS + Matrix 协议	阿里开源
Ruflo (2026-03)	Claude Code 多智能体编排	Token 成本 -75%
深即达 A2A IM (2026-03)	agent 之间的"微信"	业界首创
CLI-Anything / Agent Teams	群体智能	Claude Code 2026-06

L6 Evaluation (4 项)

项目	一句话	关键数据
孙敦灿《Agent 评测体系设计》 (2026-07-11)	22,000 字评测长文	业界最系统
3 道门槛 + 假阳性	非确定性 / 黑盒化 / 错误级联	体系化总结
4 个评测层次	Turn/Session/Trace/Outcome	必须同时看
5 大类指标	P0/P1/P2/Skill 子指标/一致性	P0 = 上线门 禁
至少一次 vs 连续成功率	生产关心连续	每组 200+ 样 本
LLM-as-a-Judge 双盲	双模型一致才算	模型 A + 模 型 B
Eval-Driven Development (EDD)	写 eval 先于写 prompt	2026 年新方 法论

L7 Self-Improvement (5 项)

项目	一句话	关键数据
RHI (Recursive Harness Self-Improvement)	Sakana AI + UC Berkeley + Stanford + Databricks	arXiv:2607.15524 / 推理成本 -60%
DeepSeek Harness (DSH)	一切皆插件	17.1万★ / 2026-08
Cordis Meta-Framework	时间/空间可组合性	6.5K★ / shigma/cordis
Shepherd	Stanford Manning 组 Git 原生 agent	2,313★ / CooperBench 28.8%→54.7%
Harness Engineering (Birgitta Böckeler, QCon 2026)	"Harness = 一切加在一起"	QCon 演讲
Harness Self-Improvement 5 步闭环	跑任务 / 收集 trace / 发现失败 / 提出新 harness / 独立 eval	本书总结

附录 B · 真实时间线 (2024-2027)

时间	事件
2024-03	Anthropic 推出 Claude 3 + Tool Use
2024-04	DSPy 开源 (Stanford NLP)
2024-05	Anthropic 《Prompt Engineering Overview》发布
2024-08	Anthropic Prompt Caching (省 90% 成本)
2024-08	OpenAI Structured Outputs
2024-10	Anthropic Agentic Tool Use 最佳实践
2024-10	OpenAI Prompt Caching
2024-11	Model Context Protocol (MCP) 推出
2025-03	OpenAI 支持 MCP
2025-04	Mem0 论文发布 (arXiv:2504.19413)
2025-06	Google Gemini 支持 MCP
2025-08	DeepSeek 支持 MCP
2025-08	DeepSeek 推出 TencentDB Agent Memory 4 层架构
2025-12	Anthropic "Skill 三层加载架构" 文档
2026-03	WebMCP (Chrome)
2026-03	深即达 A2A IM
2026-03	Ruflo (Claude Code 多智能体编排, Token -75%)
2026-04	求是 Skill 开源 (毛选方法论, 2k+ Star)
2026-05	Skills-Oriented Programming (SOP) 白皮书
2026-06	Agent Teams (Claude Code)
2026-06	Agent_Evaluation_System wiki 整理
2026-07-11	孙敦灿《Agent 评测体系设计》22,000 字

时间	事件
2026-07	腾讯程序员《Harness Engineering 团队落地规范》10 章
2026-08-17	RHI 论文发布 (arXiv:2607.15524)
2026-08	DeepSeek Harness (DSH) 17.1万★
2026-08	Shepherd 论文发布 (Stanford Manning 组)
2026-08	ai-memory (Fabio Akita)
2026-08	teamai-cli (Tencent)
2026-09	Anthropic "上下文防火墙" 内部演讲
2027-03-04	Shepherd 团队邀请作者去 Stanford workshop
2027-04-12	Stanford "Agent Harness: From Engineering to Co-evolution" workshop
2027-04-14	DSH 团队晚宴: L7 之后的范式讨论

附录 C • 术语速查

术语	解释
Harness	包裹 LLM 的"马具"——prompt + tools + memory + skills + eval + ...
Agent	Model + Harness = 能自主跑任务的 AI
Skill	把"专家知识"封装为可复用的模块 (SKILL.md 文件夹)
Subagent	拥有独立 context window 的子任务执行者
Eval	把"感觉好不好"变成"可量化、可回归"的指标体系
Self-Improvement	agent 用 eval 反馈改自己 harness 的能力
MCP	Model Context Protocol, 工具调用协议标准
KV Cache	LLM 推理的"短期记忆"基础设施
KV Cache 命中率	prompt 重复部分能被 cache 复用的比例
RAG	Retrieval-Augmented Generation, 检索增强生成
Few-shot	在 prompt 里加几个例子让模型"照葫芦画瓢"
CoT (Chain of Thought)	让模型"一步一步想"的技术
EDR / EDD	Eval-Driven Development, 写 eval 先于写 prompt
RSI	Recursive Self-Improvement, 递归自我改进
OS-level harness	操作系统层面的 agent 框架 (如 DSH)

附录 D • 引用约定

每章末尾的"引用"段引用了**项目名 / 论文 / 文档**，但**没有给 URL**——避免本书膨胀。

如果你想追溯某个项目，建议入口：

- **GitHub 项目**： github.com/ 直接搜名字
- **arXiv 论文**： arxiv.org/abs/<论文号>
- **Anthropic 文档**： docs.anthropic.com
- **OpenAI 文档**： platform.openai.com/docs
- **DeepSeek 平台**： api-docs.deepseek.com
- **公众号文章**： 在微信搜索对应关键词

完整 wiki 索引 (803 页 / 2,138 wikilinks)：

```
C:\Users\Administrator\clawd\memory\wiki\index.md
```

— 附录完 —
